SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

CSA1737 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 1 (CO1 – Unit I: Introduction to AI)

Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

Item	Details
Student name	**Saai Vardhan D**
Register number	**192525205**
Course	CSA1737 – Artificial Intelligence (L-T-P-C: 3-0-2-4)
Assignment	Assignment 1 – CO1 (Individual, 50 marks, Tasks 1–12)
Language / tools	Python 3 (3.13), Graph viz, Matplotlib; run in a Linux container (2 vCPU)
Date of submission	15 October 2026

Evidence rule followed throughout: Source code → test input → expected behaviour → actual output → performance metrics (4 input sizes, 3 start/goal pairs, 5 timing runs) → interpretation. Every number in this report was produced by the code listed in Task 8 and the Appendix; random seeds are reported.

Contents and Summary of Findings

Task	Title	Marks
1	Design the agent and the search algorithms (PEAS, formulation, BFS / UCS / IDDFS)	5
2	Identify the basic operation	2
3	Derive time- and space-complexity functions	6
4	Best-, average- and worst-case behaviour	4
5	Asymptotic analysis (O, Ω, Θ)	3
6	Formulate the recurrence relation	3
7	Solve the recurrence relation	4
8	Implement and test the algorithms	4
9	Record experimental performance	3
10	Compare theory with experimental behaviour	4
11	Recommend and defend the final design	4
12	Day-4 challenge: robot no longer knows where it is (belief-state search)	8
	Total	**50**

Key findings (all measured, graph seed 42, pairs seeds 1–3)

1. BFS is the fastest and uses the fewest nodes: at 1,000,000 states it generated 732,720 nodes in 2.21 s, but its routes take **11–24 % more travel time** than optimal because it minimises corridors, not minutes.

2. UCS (uniform-cost search) is the only strategy that always returned the minimum travel-time route (cost-optimal in every test). It generated 1.4–2.0× the nodes of BFS and took about 4× the time at 1,000,000 states (9.01 s).

3. IDDFS keeps a tiny recursion stack (154 entries at 1,000,000 states against 283,320 for BFS) and found hop-optimal routes, but re-expands nodes (about 4.6× BFS expansions) and **fails badly when the goal is unreachable** (19.7 million expansions on only 800 locations, growing about as n^3).

4. Recommended design: UCS with duplicate detection, run once from the pharmacy and cached as a shortest-path tree. BFS stays as the fast reachability pre-check.

5. Day-4 change (position unknown): the original design is **modified, not discarded**: search over belief states, add the local sensor so each percept splits the belief, and hand over to the original UCS as soon as the robot knows where it is. Without the sensor, belief states generated grew 245× from k = 2 to k = 8 (239 → 58,587); with it only 1.7× (18 → 30).

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

1.1 Understanding the problem

A multi-block hospital uses an autonomous robot to carry medicines from the central pharmacy to wards. The building is modelled as a graph G = (V, E) whose vertices are locations (junctions, corridor segments, lifts, ward doors) and whose edges are corridors with a travel time. When new wings are added the number of states n = |V| grows from 1,000 to 10,000 to 100,000 to 1,000,000+. The robot must find a route from the pharmacy to the requested ward, and the design must stay practical for both small and very large hospitals. Any preprocessing (adjacency lists, duplicate-detection structures) is part of the efficiency analysis.

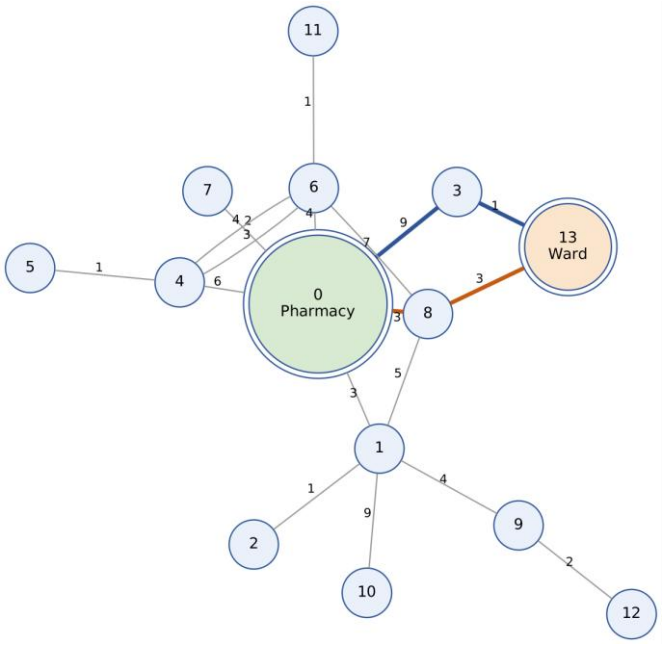


Figure 1.1 – A small example hospital graph (14 locations). Edge labels are travel times in minutes. The blue route (0–3–13, cost 10) is what BFS returns; the orange route (0–8–13, cost 6) is what UCS returns. Both have 2 corridors, but UCS is 40 % faster.

1.2 PEAS description of the robot

PEAS element	Description for the hospital medicine-delivery robot
P – Performance measure	Medicine delivered to the correct ward (correctness); minimum travel time (cost); no collisions or unsafe contact with patients and staff (safety); battery used; delivery within the medicine's time window; low planning time even when the hospital grows.
E – Environment	The hospital: pharmacy, junctions, corridors, lifts, ward doors, people, trolleys, closed doors. For route planning it is abstracted as a weighted graph G = (V, E).
A – Actuators	Drive motors and steering (move along a corridor to the next node), lift-call arm / button presser, drawer lock for medicines, horn / display for warnings.
S – Sensors	Localisation (RFID / QR waypoint tags, wheel encoders, lidar map matching), obstacle lidar and camera, door-state sensor, battery monitor, request receiver (ward number).

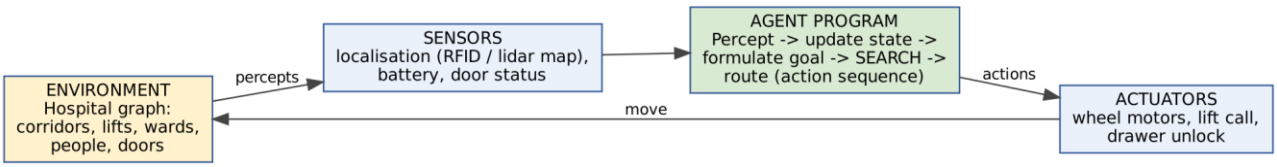


Figure 1.2 – Goal-based problem-solving agent: percepts from the hospital are turned into a goal and a search problem; the route found by search is executed as actions.

1.3 Classification of the task environment

Property	Choice	Justification
Observable	Fully observable (original problem)	The robot knows its exact node and has the complete map, so the state is fully known. This is exactly the assumption that Task 12 later removes (partially observable).
Deterministic	Deterministic	At the planning level, executing go(v, w) along a connected corridor always leads to w. Wheel slip and obstacle avoidance are handled by a low-level controller and do not change the abstract result.
Episodic / sequential	Sequential	The choice of a corridor changes where the robot is and therefore which later corridors are available; battery and time accumulate. A delivery is a sequence of dependent decisions, not independent one-shot decisions.
Static / dynamic	Static (during planning)	The map does not change while the route is being computed. People and doors do move in reality, so the robot re-plans after a closed door; at the planning level the environment is treated as static.
Discrete / continuous	Discrete	Locations form a finite set of graph nodes, actions are choices among a finite number of corridors, and time is measured in whole minutes (1–9) on each edge.
Single / multi-agent	Single-agent	Only one planning agent is considered. Other robots, staff and patients are treated as obstacles for the low-level controller, not as agents with conflicting goals in the search.

1.4 View of AI and rationality; link to early AI history

View followed: acting rationally. The robot is a rational agent: for every percept sequence it selects the action sequence that is expected to maximise its performance measure (shortest delivery time, safe, correct ward) given what it knows. It is not trying to imitate human behaviour (acting humanly / Turing test) or human thought (thinking humanly), and it does not need to derive conclusions from logic (thinking rationally); correct behaviour is what matters, and search is the means of obtaining it.

Why it is a rational problem-solving agent. It is a goal-based agent that follows the *formulate → search → execute* cycle: (i) it formulates the goal (be at ward g), (ii) it formulates the problem (states, actions, costs), (iii) it searches for a sequence of actions that reaches the goal, and (iv) it executes that sequence. Because it picks the lowest-cost route found by a complete and optimal search, no other route available to it gives a better value of the performance measure.

Early history. Finding a route through a graph is the classic *state-space search* problem on which early AI was built. The Logic Theorist (Newell, Simon and Shaw, 1956, presented at the Dartmouth workshop) and the General Problem Solver (1957–59) treated intelligence as search through a space of states using general "weak methods". Breadth-first and depth-first search were already known from graph theory, and Dijkstra's algorithm (1959), which is uniform-cost search, is the standard solution for weighted routes. The

hospital robot's route finding is therefore exactly the kind of problem for which AI problem solving was first developed.

1.5 Problem formulation

Component	Definition for the hospital robot
States	S = V = the set of locations the robot can be at (junctions, corridor segments, lifts, ward doors). |S| = n, with n ≈ 10^3, 10^4, 10^5, 10^6.
Initial state	s_0 = the pharmacy node (the robot starts from the central pharmacy).
Actions	ACTIONS(v) = { go(v, w) : (v, w) ∈ E } – move to any location directly connected to v. The average number of actions is the average degree (3.0 in our generated hospitals), so the branching factor is b ≈ 3 (≈ 2 new successors per expansion once the parent is excluded).
Transition model	RESULT(v, go(v, w)) = w. Deterministic.
Goal test	GOAL-TEST(v) is true if and only if v = g, the node of the requested ward.
Path cost	Sum of step costs; the step cost c(v, go(v, w), w) is the travel time of the corridor, an integer from 1 to 9 minutes. Therefore the minimum step cost is ε = 1. A solution is optimal if it has the least total travel time C*.

Notation used in the rest of the report: n = V = \|V\| number of states; E number of corridors; b branching factor; d depth of the shallowest goal (in corridors); m maximum depth of the state space; C* optimal path cost; ε minimum step cost; L depth limit; L* final depth limit reached.

1.6 Three meaningfully different uninformed strategies

The three strategies below differ in the **data structure that holds the frontier** (FIFO queue, priority queue, recursion stack), in **what they optimise** (fewest corridors, least travel time, fewest corridors with small stack) and in **how they are implemented** (two iterative, one recursive). Plain depth-first search was considered and rejected: on a hospital graph with loops it is incomplete without a depth limit, and it returns arbitrarily long routes. Iterative deepening keeps its low-memory idea while fixing both problems.

	Approach A – BFS	Approach B – UCS	Approach C – IDDFS
Type	Iterative (FIFO queue)	Iterative (priority queue / min-heap)	Recursive (call stack), repeated with limits 0, 1, 2, …
Basic idea	Expand the shallowest unexpanded node first, level by level.	Always expand the unexpanded node with the smallest path cost g(n) so far.	Run depth-limited DFS with limit 0, then 1, then 2 … until the goal is found; combines DFS memory with BFS completeness.
Optimal for	Number of corridors (when all steps equal)	Total travel time (any positive step cost)	Number of corridors
Complete?	Yes	Yes (ε > 0)	Yes
Preprocessing	Adjacency list; explored/parent dictionary	Adjacency list; best-cost dictionary; heap	Adjacency list; depth-seen table (duplicate detection)
Why investigated	Baseline and fastest uninformed search; shows what ignoring travel time costs.	Robot cares about minutes, not corridors; the only one of the three that guarantees minimum travel time.	Low-memory option for 1,000,000+ states; satisfies the recursive-approach requirement; gives a recurrence to analyse.

Approach A – Breadth-First Search (iterative, queue-based)

```
BFS(graph, start, goal):
    if start == goal: return [start]              # start equals goal
    parent   <- { start : None }                   # explored set + back-pointers
    frontier <- QUEUE containing start
    while frontier is not empty:
        u <- frontier.dequeue()                    # node expansion
        for each neighbour v of u in adj[u]:       # successor generation
            if v not in parent:                    # duplicate detection
                parent[v] <- u
                if v == goal: return path(parent, goal)     # goal test at generation
                frontier.enqueue(v)
    return FAILURE                                 # goal unreachable
```

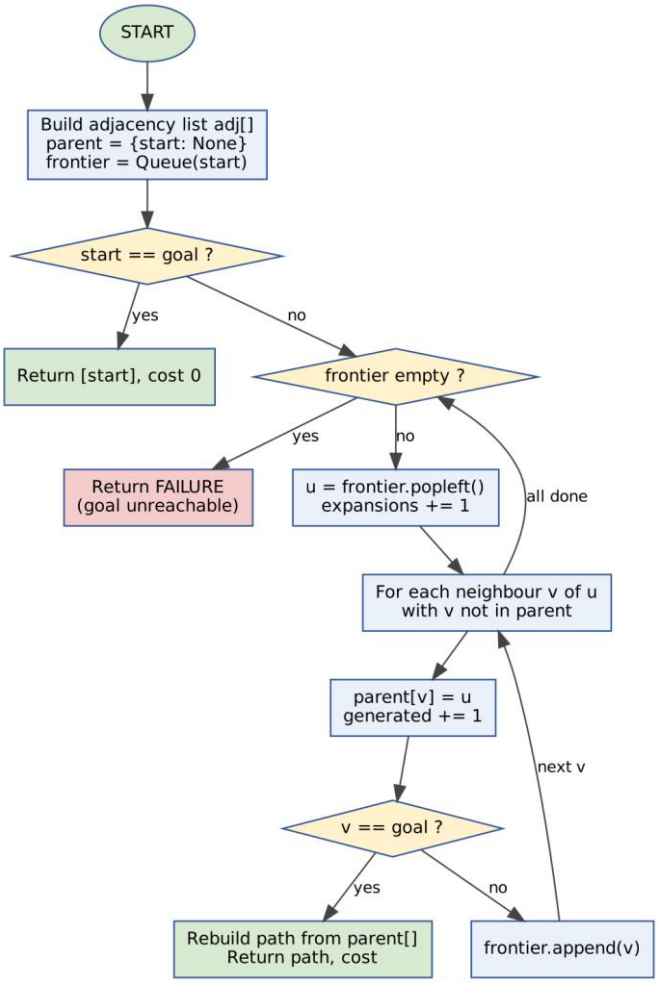


Figure 1.3 – Flowchart of BFS.

Preprocessing: build the adjacency list adj[u] = [(v, w), …] from the corridor list (each corridor stored twice, once per direction) and create an empty parent dictionary, which also acts as the explored set. **Justification:** BFS is the standard baseline; it is complete and gives the fewest-corridor route, and its explored set bounds the work by the size of the hospital.

Approach B – Uniform-Cost Search (iterative, priority-queue-based)

```
UCS(graph, start, goal):
    best     <- { start : 0 }                      # cheapest known cost to each node
    parent   <- { start : None }
    frontier <- MIN-HEAP containing (0, start)
    while frontier is not empty:
        (c, u) <- frontier.pop_min()
        if c > best[u]: continue                   # stale entry: a cheaper path was found
        if u == goal: return path(parent, goal), c # goal test at EXPANSION (needed for
optimality)
```

```
        for each (v, w) in adj[u]:
            if v not in best or c + w < best[v]:
                best[v]   <- c + w
                parent[v] <- u
                frontier.push((c + w, v))
    return FAILURE
```

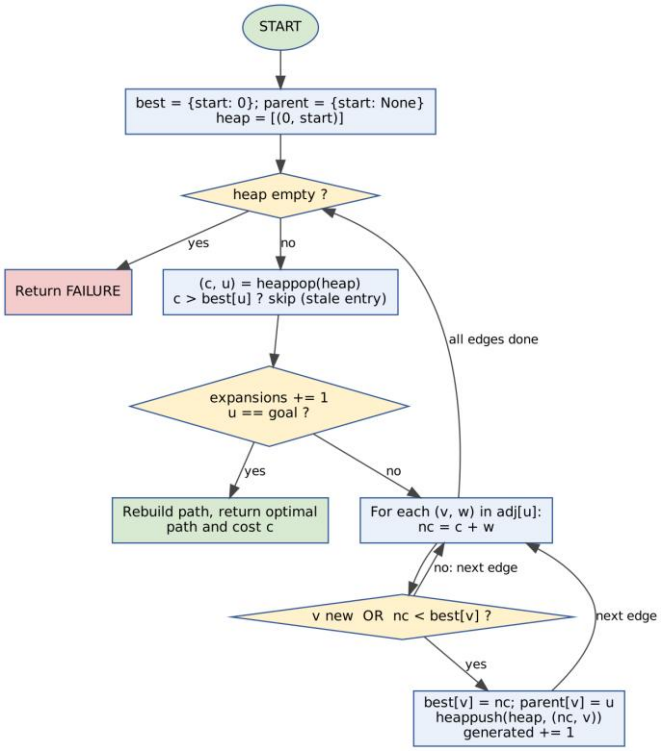


Figure 1.4 – Flowchart of UCS.

Preprocessing: adjacency list, the best-cost dictionary and a binary heap. **Justification:** the performance measure is travel time. BFS can return a two-corridor route that costs 10 minutes while another two-corridor route costs 6 (Figure 1.1); UCS is guaranteed to find the 6-minute route. The goal test is placed at expansion, not at generation, because the first time a goal is *generated* it may have been reached by a non-cheapest path.

Approach C – Iterative Deepening DFS (recursive)

```
IDDFS(graph, start, goal):
    for limit = 0, 1, 2, 3, ... :
        seen <- { start : 0 }; parent <- { start : None }
        result <- DLS(start, depth = 0, limit)
        if result == FOUND:   return path(parent, goal)
        if result == FAIL:    return FAILURE       # nothing was cut off: whole component searched
        # result == CUTOFF -> try the next, larger limit

DLS(u, depth, limit):                             # recursive depth-limited search
    expansions += 1
    if u == goal:  return FOUND
    if limit == 0: return CUTOFF
    cutoff <- false
    for each neighbour v of u:
        if v in seen and seen[v] <= depth + 1: continue   # reached before at least as shallow
        seen[v] <- depth + 1; parent[v] <- u
        r <- DLS(v, depth + 1, limit - 1)         # RECURSIVE CALL
        if r == FOUND:  return FOUND
        if r == CUTOFF: cutoff <- true
    return CUTOFF if cutoff else FAIL
```

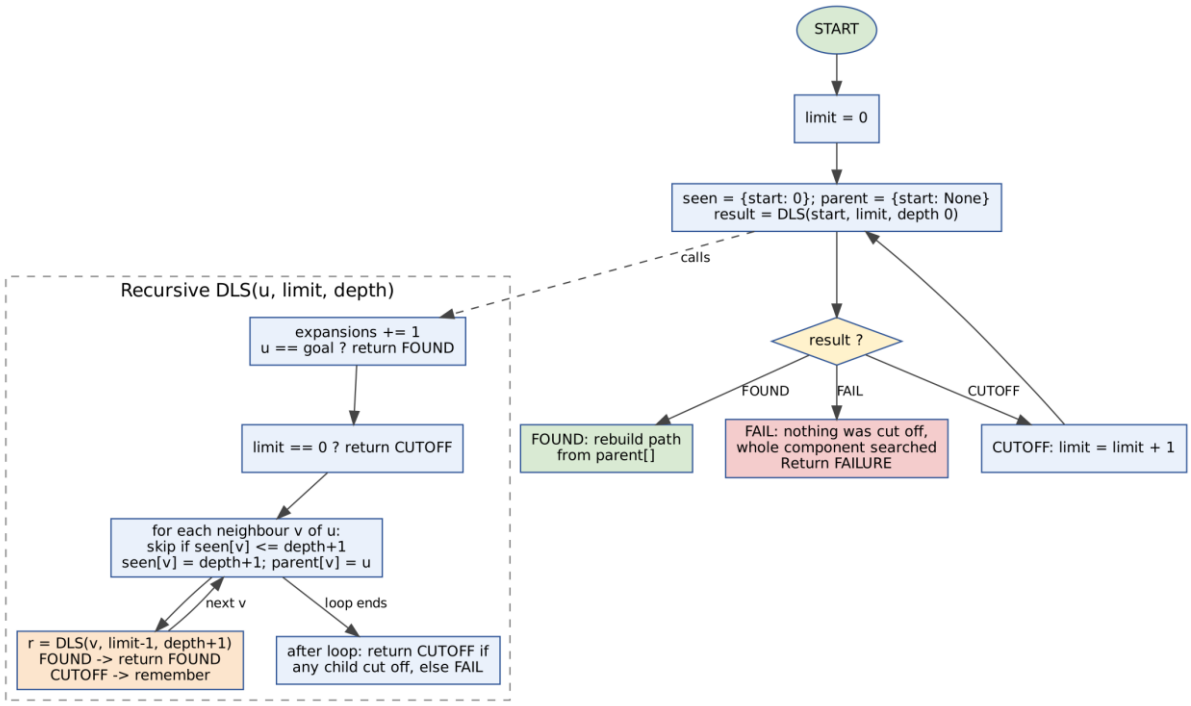


Figure 1.5 – Flowchart of IDDFS: the outer loop raises the depth limit; the dashed box is the recursive depth-limited search.

Preprocessing: adjacency list and a depth-seen table that stores the shallowest depth at which each node was reached in the current iteration. A node is expanded again only if it is reached at a strictly smaller depth, so loops in the hospital graph can never trap the search. The table is the duplicate-detection structure and costs O(V) memory (see Task 3). **Justification:** the recursion stack only holds the current path (about b·d entries), which makes IDDFS the natural candidate when the frontier of BFS becomes too large.

> **Summary of Task 1**
>
> PEAS and a six-property environment classification were given and justified; the robot follows the *acting rationally* view as a goal-based problem-solving agent; the problem was formulated with states, initial state, actions, transition model, goal test and path cost (ε = 1); three different strategies (BFS iterative, UCS iterative, IDDFS recursive) were proposed with pseudocode, flowcharts, preprocessing and justification.

TASK 2 – Identify the Basic Operation (2 Marks)

The basic operation is the operation that executes most often in the innermost part of the algorithm and therefore predominantly determines the running time. It must be (i) present in every run, (ii) proportional to the total work, and (iii) countable. In this report **node expansion** (removing a node from the frontier and examining its successors) is counted for all three algorithms, together with **nodes generated** (successors inserted into the frontier). Execution time is only a secondary measure, because it depends on the machine and on Python overhead.

Algorithm	Basic operation	Where it occurs	Why appropriate	How its count changes as the hospital grows
BFS	Node expansion: `dequeue` a node, then scan its neighbours (the `v not in parent` duplicate check and enqueue).	The body of the `while frontier` loop; the inner `for v in adj[u]` loop executes deg(u) times per expansion.	Every other action (parent insert, goal test, enqueue) happens at most once per generated node, so total work is proportional to the number of expansions × average degree.	Expansions grow with the number of nodes inside the search radius d, up to all V nodes. Measured: 479 (n = 1,000) → 504,611 (n = 1,000,000).
UCS	Node expansion plus **heap insertion** (`heappush` / `heappop`).	Pop of the cheapest node and the relaxation loop `nc < best[v]` → push.	Each push or pop costs O(log F), so the heap operation, not the plain scan, carries the extra log factor in running time.	Expands all nodes with cost ≤ C*. Measured expansions 806 → 768,379 and pushes 1,069 → 1,038,308 for the same growth.
IDDFS	A **recursive call** of DLS, i.e. one node expansion. Counted across all iterations (limit = 0 … d), so repeated expansions are included.	The first line of `DLS(u, …)`; the recursive call sits inside the neighbour loop.	Each call does a constant amount of work plus a scan of deg(u) neighbours; the total number of calls over all iterations is the real cost, including the repeated work of earlier iterations.	Grows with the ball of radius d but multiplied by the re-expansion factor. Measured 2,258 → 2,316,997 (about 4.6× the BFS expansions at every size).

Relation to algorithm behaviour. BFS expands each node at most once, so its count can never exceed V. UCS also expands each node at most once (stale heap entries are skipped without being counted), but each expansion may cause pushes into a heap, which is why its time has an extra logarithmic factor. IDDFS repeats expansions: a node at depth j is reached again in every iteration whose limit is at least j, and, because of the depth-seen table, it can also be expanded again within one iteration if a shorter path to it is found later. This is why the *count* of the basic operation for IDDFS is larger than for BFS even though both find the same route.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)

3.1 Input size parameters

Symbol	Meaning	In our generated hospitals
V = n	number of states (locations)	10^3, 10^4, 10^5, 10^6
E	number of corridors (edges)	E ≈ 1.5 n (1,499,995 at n = 10^6); sum of degrees = 2E ≈ 3 n
b	branching factor (successors per node)	average degree 3.0; effective b* measured in Task 10
d	depth of the shallowest goal (corridors)	7–15 in the experiments
m	maximum depth of the state space	up to V − 1 along a DFS path
C*	optimal path cost	28–60 minutes
ε	minimum step cost	1 minute

3.2 Approach A – BFS

Tree-search view (no duplicate detection)

Level i of the search tree has b^i nodes. In the worst case the goal is the last node generated at depth d (goal test at generation), so every node of levels 0 … d is generated:

$$N_{BFS} = 1 + b + b^2 + \ldots + b^d = (b^{d+1} - 1) / (b - 1) = \Theta(b^d)$$

If the goal test were done on expansion instead, the whole of level d would also be expanded and level d + 1 generated, giving $O(b^{d+1})$. The memory is dominated by the frontier: when the last level is being built the queue holds b^d nodes (all nodes are kept):

$$Space_{BFS} = \Theta(b^d)$$

Graph-search view (with explored set) – the one implemented

With the parent dictionary every node is enqueued at most once, so there are at most V expansions. For each expansion the loop scans deg(u) neighbours, and the sum of degrees over all nodes is 2E. Hence:

$$Expansions \leq V, \quad neighbour\ checks \leq \Sigma\, deg(u) = 2E, \quad T_{search}(BFS) = O(V + E)$$

$$Space_{BFS} = O(V) \quad (parent\ dictionary: \leq V\ entries;\ queue: \leq V\ entries)$$

Preprocessing and overall cost

Building the adjacency list inserts each of the E corridors twice (once for each direction) and allocates V lists: $T_{prep} = \Theta(V + E)$ and $Space_{prep} = \Theta(V + E)$. The overall cost is therefore

$$T_{total}(BFS) = \Theta(V + E) + O(V + E) = \Theta(V + E) \Rightarrow for\ our\ hospital,\ E = 1.5\ V:\ \approx 2.5\ V$$

Preprocessing is therefore of the same order as the search itself; for a robot that answers many requests the adjacency list is built once and reused.

3.3 Approach B – UCS

Tree-search bound. UCS expands every node whose path cost g(n) ≤ C*. Since every step costs at least ε, such a node lies at depth at most ⌊C*/ε⌋, and the successors of the nodes of that depth are also generated. Summing the levels 0 … ⌊C*/ε⌋ + 1:

$$N_{UCS} \leq 1 + b + \ldots + b^{\lfloor C^*/\varepsilon \rfloor + 1} = \Theta(b^{1+\lfloor C^*/\varepsilon \rfloor}) \quad \text{and} \quad Space_{UCS} = \Theta(b^{1+\lfloor C^*/\varepsilon \rfloor})$$

For our hospitals C* is up to 60 and ε = 1, so this bound is astronomically large (b^{61}); it shows why duplicate detection is essential. It also shows that UCS can be much worse than BFS in a tree when cheap edges form long detours (Task 4).

Graph-search bound (implemented). Each node is expanded once (a stale entry has c > best[u] and is skipped). A push happens only when an edge (u, v) improves best[v]; each directed edge is relaxed at most once (when its tail u is expanded), so there are at most 2E + 1 pushes. With a binary heap each push and pop costs O(log(2E + 1)) = O(log V):

$$T_{search}(UCS) = O((V+E)\log V), \quad Space_{UCS} = O(V+E) \ (\text{heap} \leq 2E+1 \text{ entries, best / parent} \leq V)$$

$$T_{total}(UCS) = \Theta(V+E) + O((V+E)\log V) = O((V+E)\log V)$$

For n = 10^6: (V + E) $\log_2$ V ≈ 2.5 × 10^6 × 19.93 ≈ 5.0 × 10^7 elementary heap steps in the worst case.

3.4 Approach C – IDDFS (recursive)

Tree-search view

Iteration with limit i expands every node of levels 0 … i at most once: (b^{i+1} − 1)/(b − 1) nodes. Adding the iterations 0 … d:

$$N_{IDDFS} = \Sigma_{i=0}^{d} (b^{i+1} - 1)/(b-1) = (\, b^{d+2} - b - (d+1)(b-1) \,) \,/\, (b-1)^2$$

Equivalently, a node at depth j is expanded in each of the d − j + 1 iterations whose limit is at least j:

$$N_{IDDFS} = \Sigma_{j=0}^{d} (d-j+1)\, b^j = b^d \, \Sigma_{k=0}^{d} (k+1)\, b^{-k} < b^d \cdot (b/(b-1))^2 = \Theta(b^d)$$

The re-expansion overhead compared with a single depth-d search is only the factor b/(b − 1): about 2 for b = 2 and 1.5 for b = 3 (verified numerically in Task 10: 1.995 and 1.499). The reason is that the deepest level contains most of the nodes.

Space (tree search). Only the current path is on the stack: d + 1 frames, each with at most b pending siblings, so Space = O(b·d) – linear in the depth instead of exponential.

Graph-search version (implemented, with the depth-seen table)

Within one iteration with limit L a node v is expanded only when it is reached with a strictly smaller depth than any previous visit. The stored depth takes values in {0, …, L}, so v is expanded at most L + 1 times, and an iteration expands at most V(L + 1) nodes. Summing over the iterations 0 … L*:

$$T_{search}(IDDFS) \leq V \, \Sigma_{L=0}^{L^*} (L+1) = V\,(L^*+1)(L^*+2)/2 = O(V \cdot L^{*2})$$

Here L* is the last limit used. For a **reachable** goal, L* = d (the shallowest goal depth), so T = O(V·d^2), which is polynomial and, for d ≈ log V, close to O(V $\log^2$ V). For an **unreachable** goal the loop stops only when an iteration is not cut off anywhere, i.e. when L exceeds the depth of the deepest path the DFS can walk, which may be Θ(V), not the graph diameter. Then T = O(V^3) in the worst case. (The measured values of L* and the n^3 growth are shown in Tasks 4 and 10.)

Space. Recursion stack and pending siblings: O(b·L*). The depth-seen table adds O(V) entries (it holds one depth per visited node). The stack alone is tiny, but the duplicate table makes the graph-search IDDFS O(V) in total memory; this is measured and reported honestly in Tasks 9 and 10.

Preprocessing and overall cost

$$T_{total}(IDDFS) = \Theta(V + E) + O(V \cdot L^{*2})$$

3.5 Summary of the derived functions

Algorithm	Basic-operation count (tree search)	Time (graph search, implemented)	Space (graph search)	Preprocessing	Total
BFS	$\Theta(b^d)$	O(V + E)	O(V) (frontier ≤ V)	Θ(V + E)	Θ(V + E)
UCS	$\Theta(b^{1+\lfloor C^*/\epsilon \rfloor})$	O((V + E) log V)	O(V + E) (heap ≤ 2E + 1)	Θ(V + E)	O((V + E) log V)
IDDFS	$\Theta(b^d)$, overhead b/(b − 1)	$O(V \cdot L^{*2})$; L* = d (reachable), up to V (unreachable)	stack O(b·L*) + table O(V)	Θ(V + E)	$O(V \cdot L^{*2}) + \Theta(V + E)$

Summary of Task 3

Time and space were derived by summing nodes level by level ($b^0 + \ldots + b^d$) for the tree-search forms and by counting expansions, edge relaxations and heap operations for the graph-search forms that were implemented. Preprocessing (adjacency list) is Θ(V + E) for all three and is included in the total.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

The cases are determined by *where the goal lies* relative to the start and by *whether it can be reached at all*. They were measured on one generated hospital of n = 100,000 locations (seed 42) plus a sealed ward of 4 unreachable locations, start node 5, so that the same graph is used for every algorithm.

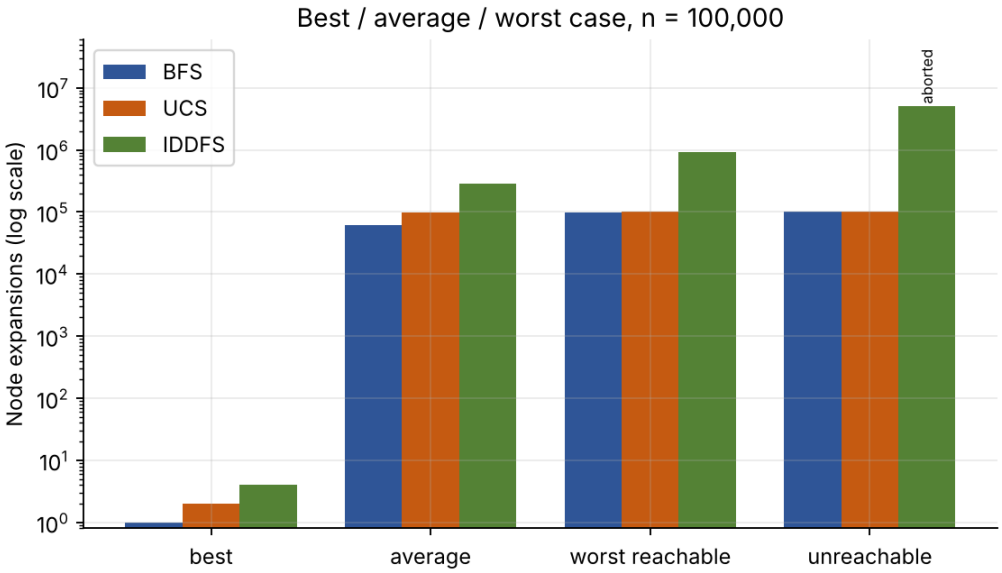


Figure 4.1 – Node expansions in the four cases at n = 100,000 (log scale). The IDDFS bar in the unreachable case is a lower bound: the run was aborted after 5,000,000 expansions.

Case	Algo	Hops	Cost	Generated	Expanded	Max frontier
best: goal is neighbour of start	BFS	1	1	3	1	1
best: goal is neighbour of start	UCS	1	1	5	2	4
best: goal is neighbour of start	IDDFS	1	1	3	4	4
average: random reachable goal (seed 1)	BFS	12	73	87,096	60,463	31,298
average: random reachable goal (seed 1)	UCS	14	60	111,965	99,270	36,981
average: random reachable goal (seed 1)	IDDFS	12	73	290,428	290,440	113
worst reachable: farthest node (depth 13)	BFS	13	75	99,956	99,434	31,467
worst reachable: farthest node (depth 13)	UCS	13	57	111,961	99,850	36,876
worst reachable: farthest node (depth 13)	IDDFS	13	75	923,302	923,315	139
worst: goal unreachable (sealed ward)	BFS	—	—	100,000	100,000	31,467
worst: goal unreachable (sealed ward)	UCS	—	—	111,969	100,000	36,876
worst: goal unreachable (sealed ward)	IDDFS	—	—	—	>5,000,000 (aborted)	—

Table 4.1 – Measured best, average and worst cases (n = 100,000).

4.1 BFS

- **Best case – goal is a neighbour of the start (d = 1).** The goal is generated while scanning the first expansion, so the search stops immediately: 1 expansion and 3 generated nodes (Ω(1)); in theory Θ(b) successors at most.

- **Average case – goal at a typical distance.** Nodes at distance < d are all expanded and, on average, about half of the nodes at distance d are generated before the goal is met. In the random hospital graph, a random pair is about 7–15 corridors apart; the three random pairs at n = 100,000 generated 55,949 nodes on average (individual pair: 87,096 generated for d = 12).
- **Worst case – goal at maximum depth or unreachable.** Every node of the component is generated and expanded: 99,956 generated / 99,434 expanded for the farthest node (depth 13) and exactly 100,000 / 100,000 for the unreachable ward. The explored set guarantees BFS stops after V expansions, so the worst case is $\Theta(V + E)$ and not $\Theta(b^d)$ (b^d would be a count of paths, mostly duplicates).

4.2 UCS

- **Best case.** The goal is the cheapest neighbour: UCS expands the start and then pops the goal as the next cheapest entry. Measured 2 expansions and 5 generated nodes.
- **Average case.** UCS expands every node whose distance is at most C*, not just the nodes on the way. In the pair-1 test it expanded 99,270 of 100,000 nodes (C* = 60); over the three pairs it expanded 90,245 on average. Because the number of nodes within cost C* grows fast, UCS usually expands *more* nodes than BFS (1.5–2.7× in the experiments) but returns the cheaper route.
- **Worst case.** (i) Unreachable goal: all 100,000 nodes expanded and 111,969 generated (11,969 extra pushes because cheaper paths to some nodes were discovered after they were already in the heap). (ii) **Cheap-edge detour:** if a long chain of ε-cost corridors leads away from the goal while the direct corridor is expensive, all of the cheap region (cost < C*) is expanded first; in the tree form this is $b^{1+\lfloor C^*/\varepsilon \rfloor}$ nodes.

4.3 IDDFS

- **Best case.** Goal adjacent to the start: limit 0 expands only the start; limit 1 expands the start and finds the goal. Measured 4 expansions (1 + 3), 3 generated.
- **Average case.** Each iteration repeats the work of the previous ones, and a node can be re-expanded in an iteration when a shorter path to it is discovered later. For the same pair (d = 12) IDDFS made 290,440 expansions against 60,463 for BFS (4.8×).
- **Worst case – reachable.** Farthest goal (depth 13): 923,315 expansions (9.2 V) against 99,434 for BFS, because the whole graph is re-explored in the last iterations with repeated re-expansion.
- **Worst case – unreachable goal.** The search can only stop when an iteration is not cut off anywhere. That requires L* to exceed the length of the longest path the DFS wanders along, not the graph diameter. In the sealed-ward tests the start's eccentricity was only 5–9 hops, but IDDFS had to raise the limit to L* = 39, 80, 181, 348 for n = 100, 200, 400, 800 (about 0.43 n). The bound $V \cdot L^{*2}/2$ from Task 3 holds and is within a factor of about 2.5 of the measured count:

n (reachable locations)	BFS expansions	UCS expansions	IDDFS expansions	Final limit L*	$V \cdot L^{*2}/2$
100	100	100	32,219	39	76,050
200	200	200	251,757	80	640,000
400	400	400	2,592,609	181	6,552,200
800	800	800	19,680,215	348	48,441,600

Table 4.2 – Unreachable goal: expansions needed to prove that no route exists (graph seed 42).

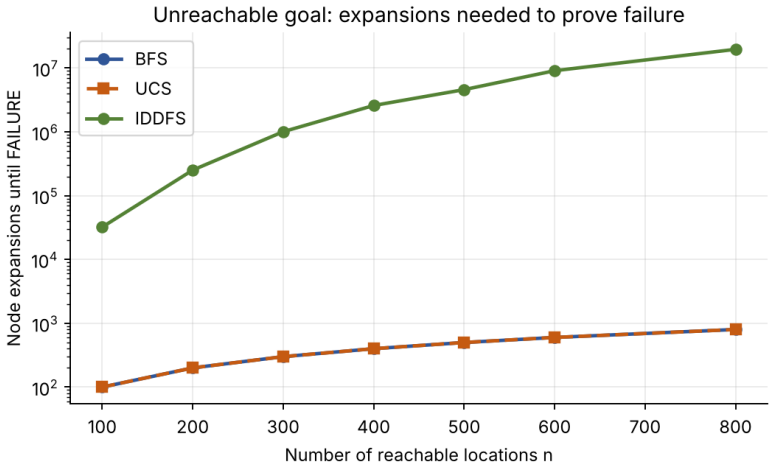


Figure 4.2 – For an unreachable goal BFS and UCS grow linearly with n, while IDDFS grows about as n^3 (100 → 800 locations: 32,219 → 19,680,215 expansions).

Why this matters for the hospital

A requested ward may be sealed off (door closed, wing under maintenance). BFS and UCS prove this in exactly V expansions; IDDFS may take millions of expansions on a graph a thousand times smaller. IDDFS is therefore unsafe as the main route finder unless a maximum depth or an expansion budget is added.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

5.1 Asymptotic forms

Algorithm	Time	Memory	Reading
BFS	Ω(1) best; O(V + E) worst; **Θ(V + E)** when the goal is farthest or unreachable	frontier + explored set: **Θ(V)** worst case (tree form Θ(b^d))	Linear in the size of the hospital; memory grows as fast as time.
UCS	Ω(log V) best; **O((V + E) log V)** worst	**O(V + E)** (heap + cost table)	Linear × logarithmic factor; same memory order as BFS but larger constants.
IDDFS	Ω(1) best; O(V·d^2) when reachable; O(V·L^{*2}) up to **O(V^3)** if unreachable	stack **Θ(b·L*)** plus a depth table O(V)	Stack grows only with the depth (logarithmically in V for these graphs); time has the extra repeated-work factor and the unreachable-goal danger.

5.2 Operation counts predicted for each hospital size

n = V	E ≈ 1.5 n	BFS: V + E	UCS: (V + E) $\log_2$V	IDDFS: V(d+1)(d+2)/2 (d = deepest test goal)	IDDFS measured (mean expansions)
1,000	1,500	2,500	24,914	45,000 (d = 8)	2,258
10,000	15,000	25,000	332,193	780,000 (d = 11)	13,907
100,000	150,000	250,000	4,152,410	9,100,000 (d = 12)	155,813
1,000,000	1,500,000	2,500,000	49,828,921	136,000,000 (d = 15)	2,316,997

Table 5.1 – Worst-case operation bounds (theory) against measured IDDFS expansions.

5.3 Interpretation for the hospital robot

- **Growth from 1,000 to 1,000,000 states (a factor of 1,000).** BFS's worst-case work grows 1,000× (linear). UCS grows about 2,000× (a factor of $\log_2 10^6 / \log_2 10^3 \approx 2$ on top of the linear growth). Measured expansions confirm roughly linear growth: slope ≈ 1.1 on log-log axes (Task 10).
- **Which limit is reached first: memory or time?** For the *tree-search* forms, memory is exhausted first: BFS and UCS keep every generated node, so memory grows as Θ(b^d) just as time does, and a machine runs out of bytes long before it runs out of patience. IDDFS in the tree form needs only O(b·d) memory but its time is exponential on a graph with loops. In the *graph-search* forms we implemented, time is the first limit that is felt in Python: BFS needed 2.2 s, IDDFS 6.6 s and UCS 9.0 s at 1,000,000 states, while the frontier of BFS (283,320 entries) and its explored dictionary (732,720 entries) are still manageable. Extrapolating the measured linear growth (an extrapolation, not a measurement), memory would start to hurt first at around 10^7–10^8 states.
- **Memory growth difference.** BFS and UCS frontier grows with n (slope ≈ 1.1), the IDDFS *stack* grows with the depth d only (154 entries at 1,000,000 states, slope 0.15). However the duplicate table of the implemented IDDFS still stores about one entry per visited node (834,438 at n = 10^6 for pair 1 against 884,195 BFS parent entries; Task 9), so the low-memory advantage of IDDFS exists only if the robot accepts the repeated work of the tree-search form.

Summary of Task 5

Time is $\Theta(V+E)$ for BFS, $O((V+E) \log V)$ for UCS and $O(V \cdot d^2)$ for IDDFS ($O(V^3)$ if the goal is unreachable); memory is $\Theta(V)$, $O(V+E)$ and stack $O(b \cdot d)$ + table $O(V)$. For a hospital growing to 10^6 states all three stay practical, but only BFS and UCS are safe when the goal may be unreachable.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

The recursive approach is the depth-limited search DLS inside IDDFS. The recurrence is derived directly from the code of `DLS(u, depth, limit)` in Task 1 (the plain tree-search form, i.e. without the depth-seen pruning, which is the form whose behaviour the recurrence describes; the effect of pruning is analysed in Task 7).

6.1 Components, read from the algorithm

Component	Identification in DLS
Size of the original problem	The remaining depth limit L (parameter `limit`). DLS(u, L) must explore all nodes up to L corridors below u.
Size of the subproblem	L − 1. Each recursive call `DLS(v, depth+1, limit-1)` has one less level of depth available.
Number of recursive calls	b: the `for` loop calls DLS once for each of the b successors v of the current node (b = branching factor).
Work outside the recursive calls	A constant c: increment the expansion counter, goal test `u == goal`, test `limit == 0`, loop bookkeeping. We count one unit per call (c = 1), so T(L) counts expansions.
Base case	L = 0: the call counts one expansion, performs the goal test, sees `limit == 0` and returns CUTOFF without any further call: T(0) = 1.

6.2 The recurrence

Let T(L) be the number of calls (node expansions) made by DLS with remaining depth L when the goal is not found (the worst case for one iteration):

$$T(L) = b \cdot T(L-1) + 1 \ \text{ for } L \geq 1, \qquad T(0) = 1$$

How the components lead to it: one call does 1 unit of own work and spawns b independent calls of size L − 1; the recursion stops at L = 0, where only the one unit of own work remains.

For IDDFS as a whole, the outer loop runs DLS for the limits 0, 1, …, d, so with $T_{ID}(d)$ the total number of expansions,

$$T_{ID}(d) = T_{ID}(d-1) + T(d), \qquad T_{ID}(0) = T(0) = 1$$

Note on the implemented (graph) version. With the depth-seen table, node v can be called at most L + 1 times in one iteration, so the recurrence is replaced by the bound $T_G(L) \leq V(L+1)$ of Task 3, which is polynomial. The recurrence above is the *worst-case behaviour without duplicate detection*; it is the reason the table is needed.

> **Summary of Task 6**
>
> Problem size L (remaining depth), subproblem size L − 1, b recursive calls, constant outside work, base case T(0) = 1 give T(L) = b·T(L − 1) + 1; for the outer iterative-deepening loop T_ID(d) = T_ID(d − 1) + T(d).

TASK 7 – Solve the Recurrence Relation (4 Marks)

7.1 Why the method fits

The recurrence T(L) = b·T(L − 1) + 1 has the "subtract-a-constant" form so the divide-and-conquer Master theorem does not apply. **Backward substitution** is the natural method; the **recursion tree** checks it and **induction** proves it.

7.2 Solution by backward substitution

$$T(L) = b \cdot T(L-1) + 1$$

$$= b \cdot [\, b \cdot T(L-2) + 1 \,] + 1 = b^2\, T(L-2) + b + 1$$

$$= b^2 \cdot [\, b \cdot T(L-3) + 1 \,] + b + 1 = b^3\, T(L-3) + b^2 + b + 1$$

$$\ldots = b^k\, T(L-k) + (b^{k-1} + \ldots + b + 1)$$

The pattern stops at k = L, where the argument reaches the base case T(0) = 1:

$$T(L) = b^L \cdot 1 + \Sigma_{i=0}^{L-1}\, b^i = b^L + (b^L - 1)/(b-1) = (b^{L+1} - 1)/(b-1) \;=\; \Theta(b^L)$$

7.3 Check by the recursion tree

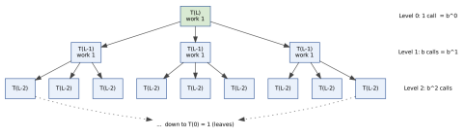


Figure 7.1 – Recursion tree of T(L) = b·T(L − 1) + 1 for b = 3. Level i has b^i calls of cost 1, so the total is $\Sigma_{i=0}^{L}\, b^i = (b^{L+1} - 1)/(b - 1)$, the same result.

7.4 Proof by induction

- **Base:** L = 0: $(b^1 - 1)/(b - 1) = 1 = T(0)$. (holds)
- **Step:** assume $T(L - 1) = (b^L - 1)/(b - 1)$. Then $T(L) = b \cdot (b^L - 1)/(b - 1) + 1 = (b^{L+1} - b + b - 1)/(b - 1) = (b^{L+1} - 1)/(b - 1)$. (holds)

7.5 What the result says

- **Dependence on depth and branching factor.** Each extra level multiplies the work by b. For b = 3, a depth-12 search needs $(3^{13} - 1)/2 = 797{,}161$ calls and depth 15 needs 21.5 million; with b = 2 and d = 12 only 8,191. The exponential growth is why a plain recursive search cannot be used on a hospital with loops and why duplicate detection (the depth-seen table, which turns the recurrence into $T_G(L) \le V(L + 1)$) is mandatory.
- **Extension to iterative deepening.** Summing the solved recurrence over the limits 0 … d:

$$T_{ID}(d) = \Sigma_{L=0}^{d}\, (b^{L+1} - 1)/(b-1) = (b^{d+2} - b - (d+1)(b-1)) / (b-1)^2 \;=\; \Theta(b^d)$$

The ratio of $T_{ID}(d)$ to the cost T(d) of one final depth-d search tends to b/(b − 1): the repeated work of the shallow iterations is only a constant factor (1.5 for b = 3, 2 for b = 2) because most nodes lie on the deepest level. Hence IDDFS has the same $\Theta(b^d)$ time as BFS (tree form) but needs only $O(b \cdot d)$ memory. These closed forms are verified exactly by experiment in Task 10 (Table 10.1).

Summary of Task 7

$T(L) = b \cdot T(L-1) + 1$, $T(0) = 1$ solves to $(b^{L+1} - 1)/(b - 1) = \Theta(b^L)$; iterative deepening multiplies this by $b/(b - 1)$ only, giving $\Theta(b^d)$ overall.

TASK 8 – Implement and Test the Algorithms (4 Marks)

8.1 Reproducible testing procedure

Item	Setting
Language / platform	Python 3.13, standard library only (random, heapq, collections, time); Linux container, 2 vCPU
Graph generator	`build_graph(n, seed, extra_ratio=0.5, sealed=0)`: random recursive tree (connected, n − 1 corridors) + 0.5 n random extra corridors (loops); travel time = random integer 1–9; optional sealed ward (separate component) for the unreachable test
Sizes	n = 1,000 → 10,000 → 100,000 → 1,000,000 (edges: 1,499 → 14,998 → 149,998 → 1,499,995)
Random seeds	**Graph seed = 42** (same graph for all three algorithms). **Start/goal pair seeds = 1, 2, 3.** Timing: 5 repeated runs on pair 1 (mean ± sd); counts averaged over the three pairs.
Same input for all	All three algorithms receive the same adjacency list and the same (start, goal) pairs.
Correctness check	`valid()` verifies that every returned path starts at the start, ends at the goal and uses only existing corridors; the optimality checks compare BFS/IDDFS hop counts with each other and UCS cost against the others.
Run command	`python3 experiments.py` (≈ 3 minutes). Source: Appendix A (experiments.py) and Section 8.2 (hospital_search.py).

8.2 Source code – hospital_search.py (all three algorithms and preprocessing)

The code corresponds one-to-one to the pseudocode of Task 1: `bfs` (iterative, queue), `ucs` (iterative, heap), `iddfs` / `dls` (recursive). The `cap` argument is an optional expansion budget used only to stop IDDFS in the unreachable-goal experiment.

```
"""
CSA17 - AI Assignment 1 : Hospital Medicine-Delivery Robot
Uninformed search on a generated hospital graph.
Author : Saai Vardhan D   |   Reg. No. : 192525205
Algorithms : (1) BFS  - iterative, FIFO queue
             (2) UCS  - iterative, priority queue (uniform-cost / Dijkstra)
             (3) IDDFS- recursive depth-limited DFS, repeated with limit 0,1,2,...
"""
import random, heapq, sys, time
from collections import deque


sys.setrecursionlimit(20000)


# ---------------------------------------------------------------------- preprocessing
def build_graph(n, seed, extra_ratio=0.5, sealed=0):
    """Hospital graph with n connected locations (+ `sealed` unreachable ones).
    Step 1: random recursive tree  -> guarantees connectivity (n-1 corridors)
    Step 2: extra_ratio*n random corridors -> loops (alternative routes)
    Step 3: optional sealed ward (separate component) for the unreachable test
    Every corridor gets an integer travel time 1..9 minutes.
    Returns adjacency list  adj[u] = [(v, w), ...]   and #edges inserted."""
    rng = random.Random(seed)
    adj = [[] for _ in range(n + sealed)]
    edges = 0
    def add(u, v, w):
        adj[u].append((v, w)); adj[v].append((u, w))
    for v in range(1, n):
        add(rng.randrange(v), v, rng.randint(1, 9)); edges += 1
    for _ in range(int(extra_ratio * n)):
        u, v = rng.randrange(n), rng.randrange(n)
        if u != v:
            add(u, v, rng.randint(1, 9)); edges += 1
    for v in range(n + 1, n + sealed):          # sealed ward: its own component
        add(v - 1, v, rng.randint(1, 9)); edges += 1
    return adj, edges


class Budget(Exception): pass
```

```
class Stats:
    def __init__(self, cap=None): self.gen = 0; self.exp = 0; self.frontier = 0; self.cap = cap   # cap = optional
expansion budget


def rebuild(parent, g):
    path, cost, v = [], 0, g
    while v is not None:
        path.append(v); p = parent[v]
        if p is None: break
        cost += p[1]; v = p[0]
    return path[::-1], cost


# ---------------------------------------------------------------------- 1. BFS
def bfs(adj, s, g):
    st = Stats(); st.gen = 1
    if s == g: return [s], 0, st
    parent = {s: None}; frontier = deque([s]); st.frontier = 1
    while frontier:
        u = frontier.popleft(); st.exp += 1
        for v, w in adj[u]:
            if v not in parent:                    # duplicate detection
                parent[v] = (u, w); st.gen += 1
                if v == g:                         # goal test at generation
                    p, c = rebuild(parent, g); return p, c, st
                frontier.append(v)
        st.frontier = max(st.frontier, len(frontier))
    return None, None, st


# ---------------------------------------------------------------------- 2. UCS
def ucs(adj, s, g):
    st = Stats(); st.gen = 1
    best = {s: 0}; parent = {s: None}; heap = [(0, s)]; st.frontier = 1
    while heap:
        c, u = heapq.heappop(heap)
        if c > best[u]: continue                   # stale entry
        st.exp += 1
        if u == g:                                 # goal test at expansion
            p, cost = rebuild(parent, g); return p, cost, st
        for v, w in adj[u]:
            nc = c + w
            if v not in best or nc < best[v]:
                best[v] = nc; parent[v] = (u, w)
                heapq.heappush(heap, (nc, v)); st.gen += 1
        st.frontier = max(st.frontier, len(heap))
    return None, None, st


# ---------------------------------------------------------------------- 3. IDDFS (recursive)
FOUND, CUTOFF, FAIL = 0, 1, 2


def dls(adj, u, g, limit, depth, seen, parent, st, cur):
    st.exp += 1
    if st.cap and st.exp > st.cap: raise Budget()
    if u == g: return FOUND
    if limit == 0: return CUTOFF
    cur[0] += len(adj[u]); st.frontier = max(st.frontier, cur[0])
    cutoff = False
    for v, w in adj[u]:
        d = depth + 1
        if v in seen and seen[v] <= d: continue   # already reached at least as shallow
        seen[v] = d; parent[v] = (u, w); st.gen += 1
        r = dls(adj, v, g, limit - 1, d, seen, parent, st, cur)
        if r == FOUND: return FOUND
        if r == CUTOFF: cutoff = True
    cur[0] -= len(adj[u])
    return CUTOFF if cutoff else FAIL


def iddfs(adj, s, g, cap=None):
    st = Stats(cap); st.gen = 1
    limit = 0
    while True:
        seen = {s: 0}; parent = {s: None}
        r = dls(adj, s, g, limit, 0, seen, parent, st, [0])
        if r == FOUND:
            p, c = rebuild(parent, g); return p, c, st
        if r == FAIL: return None, None, st        # whole component searched
        limit += 1


ALGOS = {"BFS": bfs, "UCS": ucs, "IDDFS": iddfs}


def valid(adj, path, s, g):
    if path is None: return False
    if path[0] != s or path[-1] != g: return False
    return all(any(v == b for v, _ in adj[a]) for a, b in zip(path, path[1:]))
```

8.3 Test cases: input, expected behaviour, actual output

Three required test cases were run on a hospital of 1,000 locations plus a sealed ward of 4 locations (graph seed 42): a reachable goal, an unreachable goal and start = goal.

Test input	Expected behaviour	Actual output (BFS | UCS | IDDFS)	Verdict
Reachable: pharmacy 17 → ward 640	All three return a valid route. BFS and IDDFS return a route with the fewest corridors; UCS returns the cheapest route (cost ≤ BFS cost).	BFS: 6 corridors, cost 35 (847 generated) UCS: 8 corridors, cost **18** (687 generated) IDDFS: 6 corridors, cost 35 (2,326 generated)	Pass – BFS = IDDFS = 6 hops (minimum); UCS cost 18 < 35
Unreachable: pharmacy 17 → sealed ward 1003	All three terminate and report failure (no route). BFS and UCS should expand exactly the 1,000 reachable nodes.	BFS: None, 1,000 expanded UCS: None, 1,000 expanded IDDFS: None, **38,859,637** expanded	Pass – correct, but IDDFS is 38,860× more expensive
Start = goal: pharmacy 17 → 17	Route [17], cost 0, no search needed.	BFS: path [17], cost 0, 0 expanded UCS: [17], 0, 1 expanded IDDFS: [17], 0, 1 expanded	Pass

Table 8.1 – Functional tests (full console output in Appendix B).

In addition, every one of the 36 scaling runs (4 sizes × 3 pairs × 3 algorithms) passed the `valid()` path check, and in every case BFS and IDDFS returned the same number of corridors while UCS returned a cost no larger than theirs (Table 9.1).

8.4 Console output of the test suite

```
============================================================================
TEST SUITE  (graph seed 42, n = 1000 + sealed ward of 4 locations)
============================================================================


Test: reachable (pharmacy 17 -> ward 640)
  BFS   path_len=   6 cost=  35 gen=  847 exp=  566 frontier= 344 valid/expected=True
  UCS   path_len=   8 cost=  18 gen=  687 exp=  329 frontier= 347 valid/expected=True
  IDDFS path_len=   6 cost=  35 gen= 2326 exp= 2332 frontier=  46 valid/expected=True


Test: unreachable (pharmacy 17 -> sealed ward 1003)
  BFS   path_len=None cost=None gen= 1000 exp= 1000 frontier= 344 valid/expected=True
  UCS   path_len=None cost=None gen= 1114 exp= 1000 frontier= 373 valid/expected=True
  IDDFS path_len=None cost=None gen=38859200 exp=38859637 frontier=1792 valid/expected=True


Test: start == goal (pharmacy 17 -> 17)
  BFS   path_len=   0 cost=   0 gen=    1 exp=    0 frontier=   0 valid/expected=True
  UCS   path_len=   0 cost=   0 gen=    1 exp=    1 frontier=   1 valid/expected=True
  IDDFS path_len=   0 cost=   0 gen=    1 exp=    1 frontier=   0 valid/expected=True
```

TASK 9 – Record Experimental Performance (3 Marks)

For each size the three algorithms were run on the same graph and the same three (start, goal) pairs. Nodes generated, nodes expanded and maximum frontier are the mean of the three pairs; depth and cost are listed for each pair; time is the mean ± standard deviation of **5 runs** on pair 1. "Opt." shows which optimality property the returned route satisfied in all three pairs: hop-optimal (fewest corridors) or cost-optimal (least travel time).

n = 1,000 locations (edges 1,499, average degree 3.00, preprocessing = building the adjacency list: 0.001 s)

Algorithm	Generated	Expanded	Max frontier	Depth d (3 pairs)	Path cost (3 pairs)	Opt.	Time (s), 5 runs
BFS	709	479	285	8, 7, 8	37, 33, 43	hop-optimal only	0.0004 ± 0.0001
UCS	1,069	806	358	11, 7, 12	34, 33, 35	cost-optimal only	0.0009 ± 0.0001
IDDFS	2,250	2,258	53	8, 7, 8	37, 33, 43	hop-optimal only	0.0018 ± 0.0000

n = 10,000 locations (edges 14,998, average degree 3.00, preprocessing = building the adjacency list: 0.016 s)

Algorithm	Generated	Expanded	Max frontier	Depth d (3 pairs)	Path cost (3 pairs)	Opt.	Time (s), 5 runs
BFS	4,096	2,790	1,649	7, 7, 11	36, 32, 60	hop-optimal only	0.0005 ± 0.0002
UCS	6,302	4,160	2,801	8, 10, 12	28, 29, 46	cost-optimal only	0.0030 ± 0.0003
IDDFS	13,899	13,907	78	7, 7, 11	36, 32, 60	hop-optimal only	0.0010 ± 0.0001

n = 100,000 locations (edges 149,998, average degree 3.00, preprocessing = building the adjacency list: 0.189 s)

Algorithm	Generated	Expanded	Max frontier	Depth d (3 pairs)	Path cost (3 pairs)	Opt.	Time (s), 5 runs
BFS	55,949	33,602	23,902	12, 8, 11	73, 46, 62	hop-optimal only	0.1471 ± 0.0127
UCS	109,606	90,245	36,974	14, 8, 15	60, 46, 53	cost-optimal only	0.5116 ± 0.0192
IDDFS	155,802	155,813	103	12, 8, 11	73, 46, 62	hop-optimal only	0.4521 ± 0.0056

n = 1,000,000 locations (edges 1,499,995, average degree 3.00, preprocessing = building the adjacency list: 2.840 s)

Algorithm	Generated	Expanded	Max frontier	Depth d (3 pairs)	Path cost (3 pairs)	Opt.	Time (s), 5 runs
BFS	732,720	504,611	283,320	13, 15, 12	78, 76, 66	hop-optimal only	2.2091 ± 0.1093

Algorithm	Generated	Expanded	Max frontier	Depth d (3 pairs)	Path cost (3 pairs)	Opt.	Time (s), 5 runs
UCS	1,038,308	768,379	368,627	16, 20, 14	59, 58, 60	cost-optimal only	9.0114 ± 0.2147
IDDFS	2,316,984	2,316,997	154	13, 15, 12	78, 76, 66	hop-optimal only	6.5635 ± 0.5001

Table 9.1 – Experimental performance for the four state-space sizes (graph seed 42, pair seeds 1, 2, 3).

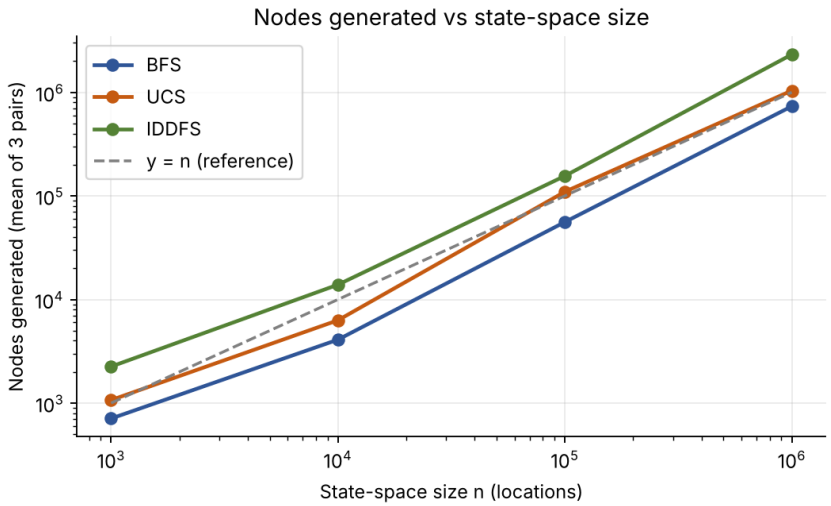


Figure 9.1 – Nodes generated against state-space size (log–log). The dashed line is y = n.

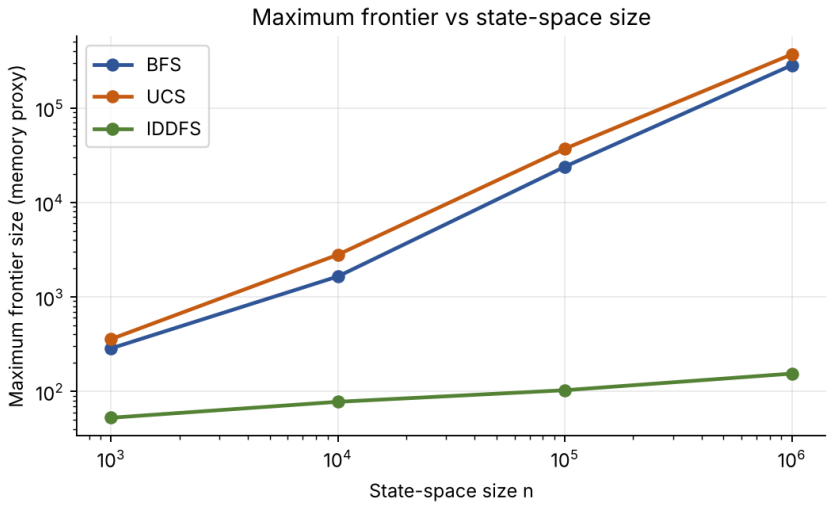


Figure 9.2 – Maximum frontier size (memory proxy). BFS and UCS grow with n; the IDDFS stack grows only with the depth.

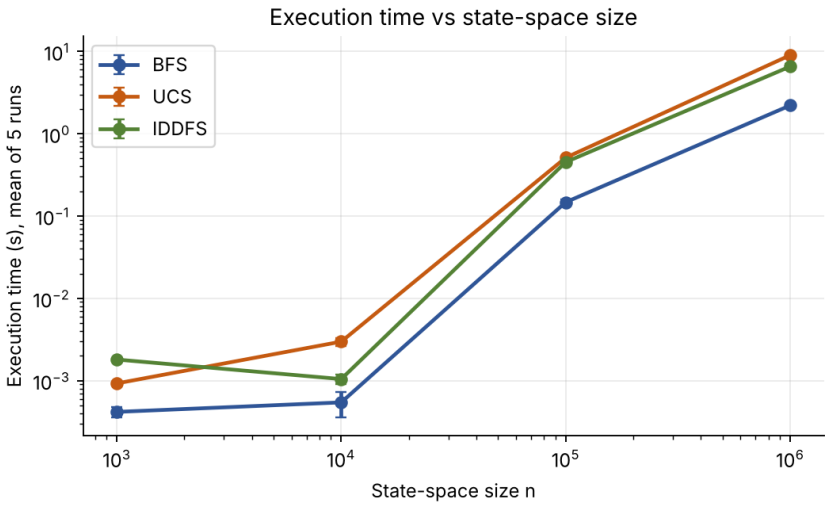


Figure 9.3 – Execution time (mean of 5 runs with standard-deviation bars). UCS is slowest because of heap operations; BFS fastest.

Route quality – cost of ignoring travel time

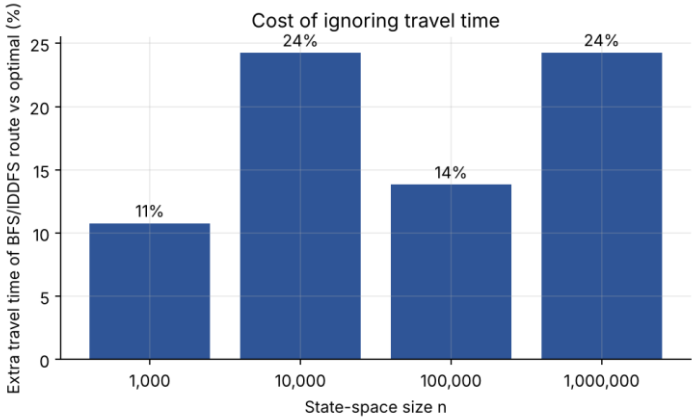


Figure 9.4 – Extra travel time of the BFS / IDDFS routes compared with the UCS (optimal) routes, summed over the three pairs of each size.

Memory of the implemented IDDFS (stack versus duplicate table)

n	Depth d (pair 1)	IDDFS stack frontier	IDDFS depth-seen table entries	BFS parent entries (generated)
1,000	8	56	946	965
10,000	7	69	751	1,607
100,000	12	113	71,351	87,096
1,000,000	13	183	834,438	884,195

Table 9.2 – For pair 1: the recursion stack is tiny, but the duplicate-detection table holds about one entry per visited node, so the implemented graph IDDFS is not smaller in memory than BFS.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

10.1 Growth of nodes generated and of frontier size

From $n = 10^4$ to $n = 10^6$ (a factor of 100) the log-log slope of each measured quantity is slope = $\log_{10}(\text{ratio})/2$; a slope of 1 means exactly linear growth, as theory predicts for graph search:

Quantity	BFS	UCS	IDDFS	Theory (graph search)
Nodes generated (×100 in n)	178.9× (slope 1.13)	164.7× (slope 1.11)	166.7× (slope 1.11)	O(V + E): slope 1
Nodes expanded	180.9× (1.13)	184.7× (1.13)	166.6× (1.11)	≤ V (BFS, UCS); $O(V \cdot d^2)$ (IDDFS)
Maximum frontier	171.8× (1.12)	131.6× (1.06)	2.0× (0.15)	Θ(V), Θ(V), O(b·d)
Time, $n = 10^5 \rightarrow 10^6$ (×10)	15.0× (1.18)	17.6× (1.25)	14.5× (1.16)	V; V log V; $V \cdot d^2$

Table 10.1a – Observed growth between $n = 10^4$ and $n = 10^6$.

- **Agreement.** Nodes generated, expanded and the BFS/UCS frontier all grow almost linearly (slope 1.06–1.13), which agrees with Θ(V + E). The IDDFS frontier barely grows (×2.0 for ×100 in n), which agrees with O(b·d) because d grows only logarithmically in n (about 8 → 13). UCS time grows fastest (slope 1.25), which agrees with the extra log factor and the heap.
- **Slope above 1.** The slopes are slightly above 1 because the depth d of random pairs grows with n (7–8 corridors at 10^3, 12–15 at 10^6) and the search covers a larger share of the graph when d is larger. Time slopes (1.16–1.25) are larger still because of cache misses and dictionary growth at 10^6 entries.

10.2 Effective branching factor: theory b^d versus graph search

Tree-search theory predicts b^d nodes. With average degree 3 (b ≈ 2 new successors) and d = 13 it would give only $2^{14} \approx 16{,}000$ nodes, but BFS generated 732,720. The effective branching factor $b^* = (\text{generated})^{1/d}$ is:

n	mean depth d	BFS generated	b* = generated^(1/d)
1,000	7.67	709	2.35
10,000	8.33	4,096	2.71
100,000	10.33	55,949	2.88
1,000,000	13.33	732,720	2.75

The effective branching factor is about 2.4–2.9 and not constant, because the random hospital has hub nodes of high degree (the recursive-tree construction creates hubs), and the count is capped by n because duplicates are rejected. The closed forms such as $(b^{d+1} - 1)/(b - 1)$ therefore describe the *shape* of the growth, while the graph-search bound Θ(V + E) predicts the *magnitude* at large d.

10.3 Verification of the recurrence and of the IDDFS overhead

To check Tasks 6–7 without the interference of duplicates, the plain recursive DLS (no explored set) was run on perfect b-ary trees with the goal at the last leaf (worst case). The measured number of calls T(L) matches $(b^{L+1} - 1)/(b - 1)$ exactly for every L:

L	0	1	2	3	4	5	6	7
b = 2: measured T(L)	1	3	7	15	31	63	127	255

L	0	1	2	3	4	5	6	7
b = 2: (b^(L+1) − 1)/(b − 1)	1	3	7	15	31	63	127	255
b = 3: measured T(L)	1	4	13	40	121	364	1,093	3,280
b = 3: (b^(L+1) − 1)/(b − 1)	1	4	13	40	121	364	1,093	3,280

Table 10.1 – Recurrence T(L) = b·T(L−1) + 1: measured calls versus the closed-form solution (full table in Appendix B).

For the whole iterative-deepening run the measured totals were 4,083 calls (b = 2, d = 10) and 4,916 calls (b = 3, d = 7), exactly equal to the formula $(b^{d+2} - b - (d + 1)(b - 1))/(b - 1)^2$; the ratio to one final depth-d search was 1.995 and 1.499 against the predicted b/(b − 1) = 2 and 1.5. **Theory and experiment agree exactly where the assumptions of the theory (a tree) hold.**

10.4 Which algorithm scales better and where theory and observation differ

Observation	Value	Explanation
IDDFS / BFS expansions	4.7, 5.0, 4.6, 4.6 (n = 10^3 … 10^6)	Not the tree-theory factor b/(b − 1) ≈ 1.5: in a graph a node is re-expanded in every iteration (limit ≥ its depth) and also within an iteration whenever a shorter route to it is found later (depth-seen rule). The factor is stable with n, so IDDFS has the same growth rate as BFS but a larger constant.
UCS / BFS expansions and generated	1.5–2.7× and 1.4–2.0×	UCS must expand all nodes with cost ≤ C* (about 77 % of the graph at n = 10^6), BFS only those with fewer than d hops.
UCS / BFS time	2.2, 5.5, 3.5, 4.1×	Heap push/pop is O(log F) and slower than deque operations: 10.4 µs per UCS expansion against 3.5 µs for BFS (n = 10^6, pair 1).
Time per expansion, IDDFS	2.3 µs	Recursive calls are cheap (no queue or heap), so IDDFS does 4.6× the expansions in only 3× the time of BFS.
Tie-breaking	—	Heap order for equal costs follows node numbers; DFS order follows adjacency order. These change which optimal route is returned and the exact count at the margin, but not the growth rates.
Graph structure and language overhead	—	Hub nodes (high degree) raise the effective branching factor; Python dictionaries, garbage collection and cache effects at 10^6 entries raise the time slope above the theoretical 1.0.

Conclusion of Task 10

Scaling: BFS scales best in nodes and time; IDDFS scales best in frontier (stack) but not in duplicate-table memory and only when the goal is reachable; UCS scales with an extra log factor but is the only cost-optimal one. The measured growth (slope ≈ 1.1) agrees with the theoretical Θ(V + E) family; the differences are explained by depth growth, hubs, duplicates, tie-breaking and Python overhead.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

11.1 Recommendation

> **Recommended design**
>
> **Uniform-cost search (UCS, Dijkstra) with duplicate detection**, run **once from the pharmacy** (the start of every delivery) over the whole hospital graph, and its parent pointers cached as a shortest-path tree. Every later delivery request is answered by following parent pointers from the ward back to the pharmacy in O(path length) time. The tree is recomputed only when the map changes (door closed, wing opened). **BFS** is kept as a cheap reachability pre-check and as the fallback when every corridor takes equal time.

11.2 Why UCS is more appropriate than the alternatives (own evidence)

Criterion	BFS	UCS	IDDFS	Evidence
Minimum travel time (the performance measure)	No: 11–24 % extra minutes	Yes: always optimal	No: same route as BFS	Table 9.1 costs; Figure 9.4; test: cost 35 vs 18
Terminates fast on an unreachable ward	Yes: V expansions	Yes: V expansions	No: 19.7 M expansions at n = 800	Table 4.2
Nodes / time at n = 10^6	732,720 / 2.2 s	1,038,308 / 9.0 s	2,316,984 / 6.6 s	Table 9.1
Frontier memory	283,320	368,627	154 (stack) + table ≈ 834,000	Tables 9.1, 9.2
Theoretical growth	Θ(V+E)	O((V+E) log V)	O(V·d²) … O(V³)	Task 3

Why not simply choose BFS, which has the lowest Big-O? Because its lower complexity buys a *different answer*: BFS minimises corridors, not minutes. Over the three pairs it returned routes 10.8 %, 24.3 %, 13.8 % and 24.3 % more expensive than UCS (n = 10^3, 10^4, 10^5, 10^6). A robot delivering time-critical medicine cannot accept an avoidable 24 % delay, and the extra planning cost of UCS can be paid once. **Why not IDDFS?** Its only strength, the tiny recursion stack, is cancelled by the duplicate table that it needs to be usable on a graph with loops (Table 9.2), and its running time is 4.6× BFS and unbounded in practice for unreachable goals.

11.3 Efficiency, preprocessing cost, trade-offs and limitations

- **Theoretical efficiency:** O((V + E) log V) time, O(V + E) memory.
- **Observed:** at n = 10^6 one full UCS query expanded 768,379 nodes (77 % of the graph) in 9.0 s of pure Python; frontier 368,627. At n = 10^5 a query took 0.51 s.
- **Preprocessing:** building the adjacency list took 2.84 s at n = 10^6 (0.19 s at 10^5). Because the start is always the pharmacy, one UCS run costs about the same as a single query (an unreachable-goal run expands exactly all 100,000 nodes at n = 10^5, only 1.1× the average query's 90,245 expansions), so caching the whole tree is almost free and makes each later request effectively constant time.
- **Trade-offs and limitations:** (i) UCS uses no knowledge of where the goal is, so it expands about 77 % of the graph even for a nearby ward when the whole tree is not needed; an informed search such as A* with a straight-line distance heuristic (Unit II) would expand far fewer nodes. (ii) Heap operations make it about 4× slower than BFS in Python; a compiled implementation would reduce all times by a large constant factor. (iii) Memory is linear in V. (iv) The tree must be recomputed when a corridor is closed.

11.4 Does it remain practical as the hospital grows?

Yes. Measured time grew with a slope of about 1.25 per decade (0.51 s at 10^5 to 9.0 s at 10^6), consistent with O((V + E) log V). Extrapolating this trend linearly (an extrapolation, not a measurement), a 10^7-state hospital would need on the order of 2–3 minutes in Python to build the tree once, and memory would be about ten times the 10^6 figure; both are acceptable for an offline computation that is repeated only when the map changes. The design is therefore practical for 1,000 to 1,000,000+ states.

> **Summary of Task 11**
>
> UCS with duplicate detection and a cached shortest-path tree is recommended because it is the only strategy that guarantees minimum travel time, it stops after V expansions even if a ward is unreachable, and its measured cost grows almost linearly with the hospital; BFS is the faster fallback, IDDFS is rejected.

TASK 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

Changed requirement. After a power restart or sensor fault the robot no longer knows exactly where it is. It knows only that it is at one of k possible junctions, and it must still reach the pharmacy docking station, whichever junction it is really at. **Decision: MODIFY.** The original UCS/BFS engine is kept, but a belief-state layer is added in front of it, and a local sensor is used so that the robot localises itself as quickly as possible; the original algorithm then finishes the job.

A. Impact analysis (2 marks)

What has changed and which assumptions break

Original assumption	Original problem	Day-4 problem	Effect
Observability	Fully observable: the robot knows its node	**Partially observable:** it knows only a set of k possible nodes	The state is no longer a single node. The goal test and the choice of actions can no longer depend on "the" current node.
Initial state	One known start state s_0 (pharmacy)	An **initial belief** $b_0 = \{x_1, \ldots, x_k\}$	The start of search is a set of states.
Goal	Reach a ward node g from the pharmacy	Reach the **pharmacy dock** from *every* possible position	The goal is satisfied only when the whole belief is inside the goal: $b \subseteq \{\text{Pharmacy}\}$. The robot must be *certain* it has arrived.
Plan	A path (sequence of nodes)	A **conformant plan** (without sensors) or a **conditional plan** (with sensors)	One action sequence must work for all k candidates, or branch on percepts.
Determinism	Deterministic	Still deterministic in the world, uncertain in knowledge	The uncertainty is only about *where* the robot is; the sensorless problem is deterministic in belief space.

Why the robot's knowledge becomes part of the state

Two situations in which the robot stands at the *same* physical junction but believes it could be at different sets of junctions require different actions. For example, if the robot is at J2 and it knows it is at J2, "go West" is a good move; if it believes it is in {J2, J6} the same move could take it away from the pharmacy in the J6 case. The best action depends on what the robot knows, not only on where it is. Therefore the state of the *search* problem must be the robot's knowledge: a **belief state** $b \subseteq S$, the set of physical states that are consistent with everything the robot has done and sensed.

B. Evaluation of the original design (2 marks)

- **Can the original algorithm still be applied?** Yes, as an engine. BFS/UCS work on any problem with states, actions, a transition function and a goal test. We apply them to a new problem whose **state is a belief state**, whose action result is RESULT(b, a) = { result(s, a) : $s \in b$ }, and whose goal test is $b \subseteq \{\text{pharmacy}\}$. What cannot be kept is the *old state definition* (one node) and the idea of finding one path: running k independent routes from each candidate junction would give k conflicting instructions and the robot would not know which to follow.
- **How large can the new search space be?** There are $2^N - 1$ non-empty subsets of the N physical states. For the 6 × 6 test floors (N = 28–31 open junctions) that is 2^{28}–$2^{31} - 1$ = 268 million to 2.1 billion belief states, against only 29 physical states; for a hospital of $N = 10^6$ it would be $2^{1{,}000{,}000}$:

impossible to enumerate. So the *naive* use of the original design on the full belief space is **not acceptable**.

- **But the reachable part is much smaller.** In a deterministic world an action maps a belief to a set that is never larger ($|b'| \le |b|$), so from an initial belief of size k only subsets of size ≤ k can ever appear. Their number is $\Sigma_{j=1}^{k} C(N, j) = O(N^k)$: polynomial in N for fixed k, but growing fast with k. For N = 29 this bound is 435 (k = 2), 27,840 (k = 4) and 6,474,540 (k = 8). The measured numbers of belief states generated (Table 12.2) stay below these bounds, but still grow very fast with k.

C. Adaptation: the revised strategy (2 marks)

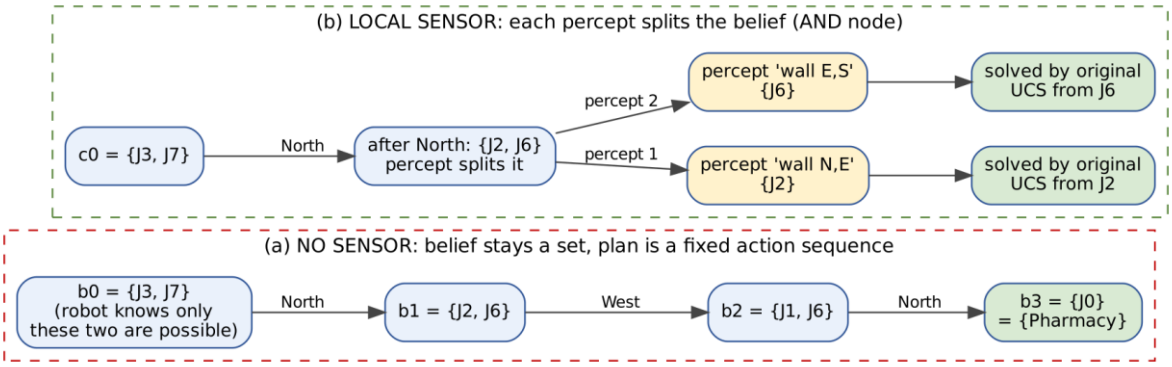


Figure 12.1 – Illustrative belief-state evolution (junction names are examples). (a) Without a sensor the belief stays a set and the plan is a fixed action sequence. (b) With a local sensor each percept splits the belief into smaller sets; singletons are handed to the original UCS.

Strategy 1 – Sensorless search over belief states (conformant plan)

BFS over belief states: start node b_0; successors RESULT(b, a) for the four moves N, E, S, W (a move into a wall leaves the robot where it is, so sets merge and shrink); goal when b = {Pharmacy}. The solution is one fixed action sequence that works whatever the true start was. Because the robot never observes anything, it can only **coerce** the possibilities together by pushing against walls, which makes plans longer than the known-location shortest path.

Strategy 2 – Search with a local sensor and a conditional plan (adopted)

After each move the robot senses a local percept (here, which of its four sides are blocked). The belief splits into one sub-belief per percept: the plan becomes a tree (an AND–OR tree): at an **OR node** the robot chooses an action; at an **AND node** the plan must work for *all* percepts that can occur. Iterative deepening on the plan depth finds the shortest worst-case plan. As soon as a sub-belief becomes a **singleton** the robot knows where it is and the **original UCS** (Task 11) takes over: this is the sense in which the original design is **modified, not discarded**.

Effect on complexity

Quantity	Original design	Sensorless belief search	With local sensor
State	node, $\|S\| = N$	subset of nodes; reachable states ≤ $\Sigma_{j\le k} C(N, j) = O(N^k)$	subset sharing one percept; smaller, because every percept splits the set
Time	$O((V + E) \log V)$	$O(\|B_{reach}\| \cdot 4)$ with $\|B_{reach}\|$ up to $O(N^k)$	tree of splits; measured 18–30 belief states for k ≤ 8
Plan	path of d corridors	one fixed sequence, longer than d	conditional plan whose worst-case length equals the known-location d in our tests

D. Evidence-based defence (2 marks)

Experiment. Five random 6 × 6 floors (seeds 1–5, about 18 % blocked cells, N = 28–31 open junctions, pharmacy dock at the top-left cell). For k = 2, 4, 8 a random set of k start junctions was drawn (seed = 100k + floor seed). Three variants were solved: (1) known location (original design, reference plan length = worst start), (2) **no sensor** (BFS over belief states, cap 2,000,000), (3) **with local sensor** (AND–OR search with iterative deepening on plan depth). Code and output follow.

k	Known-location plan (d)	No sensor: belief states generated	No sensor: plan length	With sensor: belief states generated	With sensor: plan length (worst case)
2	6.2	239	7.0	18	6.2
4	7.8	7,929	10.2	27	7.8
8	10.0	58,587	11.6	30	10.0

Table 12.1 – Mean over the five floors. Belief states generated and plan length for k = 2, 4, 8 with and without the local sensor.

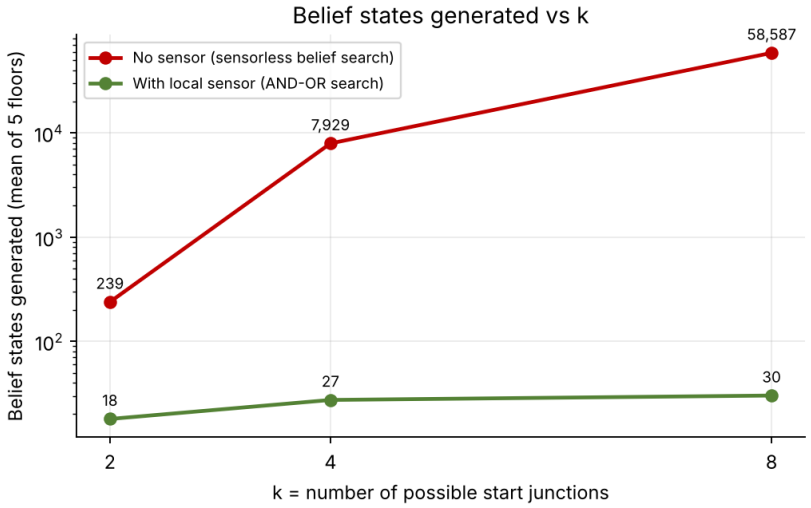


Figure 12.2 – Belief states generated against k (log scale).

Per-floor results

k	Floor seed	Open cells N	Known plan	No sensor: generated	No sensor: plan	Sensor: generated	Sensor: plan	Bound Σ C(N, j≤k)
2	1	29	9	328	9	18	9	435
2	2	29	7	398	9	29	7	435
2	3	31	3	51	3	14	3	496
2	4	28	7	280	8	16	7	406
2	5	28	5	136	6	13	5	406
4	1	29	8	9,808	11	29	8	27,840
4	2	29	9	5,283	9	28	9	27,840
4	3	31	6	7,768	9	25	6	36,456
4	4	28	7	13,338	12	26	7	24,157
4	5	28	9	3,447	10	29	9	24,157
8	1	29	9	101,873	12	31	9	6,474,540
8	2	29	10	49,158	10	32	10	6,474,540
8	3	31	10	45,840	10	28	10	11,460,948

k	Floor seed	Open cells N	Known plan	No sensor: generated	No sensor: plan	Sensor: generated	Sensor: plan	Bound Σ C(N, j≤k)
8	4	28	8	69,889	12	29	8	4,791,322
8	5	28	13	26,175	14	31	13	4,791,322

Table 12.2 – All 15 experiments. In every row the number of belief states generated is below the theoretical bound Σ C(N, j), j ≤ k.

Analysis

- **Belief space growth without a sensor:** the mean number of belief states generated rises from 239 (k = 2) to 7,929 (k = 4) to 58,587 (k = 8): a factor of 245 for a 4× increase in k, in line with the $O(N^k)$ bound. Plan lengths rise from 7.0 to 10.2 to 11.6 steps, against 6.2, 7.8 and 10.0 for the known-location reference, i.e. 1–2.4 steps of extra "coercion" moves are needed.
- **With the local sensor:** only 18, 27 and 30 belief states are needed, a growth of 1.7× from k = 2 to k = 8, and the worst-case plan length equals the known-location reference (6.2, 7.8, 10.0) in all 15 experiments. Each percept splits the belief, so sets become singletons quickly.
- **Defence of the decision:** the mathematical bound shows that the original single-state design is insufficient (belief space is exponential), and the experiment shows that adding a local sensor and searching over beliefs keeps the problem small (≤ 32 belief states) with plans no longer than before. Therefore the correct decision is to **modify** (belief-state layer + sensor, with original UCS after localisation) rather than retain or fully redesign.
- **Limitations:** the model is a small 6 × 6 floor and the percept is a 4-bit wall pattern; equality of the sensor plan with the known-location plan may partly be due to the small size and the distinct local patterns. Floors with many identical-looking corridors would need more steps, and the sensorless search can exceed memory for k > 8 (the bound for k = 8, N = 29 is already 6.5 million).

Source code – belief_search.py

```
"""
CSA17 - AI Assignment 1, Task 12 : robot that only knows it is at one of k junctions.
Author : Saai Vardhan D | Reg. No. 192525205
Ward floor = R x C grid of junctions with some blocked cells.  Actions N,E,S,W (a move into a
wall leaves the robot where it is).  Goal = pharmacy docking station at the top-left cell.
 (a) NO SENSOR   : search over BELIEF STATES (sets of possible junctions) -> sensorless plan (a fixed action sequence)
 (b) LOCAL SENSOR: after each move the robot senses which of its 4 sides are blocked (local percept);
                   the belief splits by percept -> AND-OR search gives a CONDITIONAL plan.
"""
import random, itertools, statistics, json
from collections import deque


ACT = {"N": (-1, 0), "E": (0, 1), "S": (1, 0), "W": (0, -1)}


def make_floor(R, C, seed, block=0.18):
    rng = random.Random(seed)
    while True:
        open_ = {(r, c) for r in range(R) for c in range(C) if rng.random() > block}
        open_.add((0, 0))
        seen, st = {(0, 0)}, [(0, 0)]                    # keep only cells connected to the pharmacy
        while st:
            r, c = st.pop()
            for dr, dc in ACT.values():
                n = (r + dr, c + dc)
                if n in open_ and n not in seen: seen.add(n); st.append(n)
        if len(seen) >= 0.6 * R * C: return seen


def move(cells, a):
    dr, dc = ACT[a]
    def f(p):
        q = (p[0] + dr, p[1] + dc); return q if q in cells else p
    return f


def percept(cells, p):                                   # 4-bit wall pattern N,E,S,W
    return tuple((p[0] + dr, p[1] + dc) not in cells for dr, dc in ACT.values())


GOAL = (0, 0)
```

```
def sensorless(cells, start_set, cap=2_000_000):
    """BFS over belief states.  Returns (plan length, belief states generated) or (None, generated)."""
    start = frozenset(start_set); seen = {start}; q = deque([(start, 0)]); gen = 1
    while q:
        b, d = q.popleft()
        if b == {GOAL}: return d, gen
        for a in ACT:
            nb = frozenset(map(move(cells, a), b))
            if nb not in seen:
                seen.add(nb); gen += 1; q.append((nb, d + 1))
                if gen > cap: return None, gen
    return None, gen


def with_sensor(cells, start_set, max_depth=40):
    """AND-OR search (iterative deepening on plan depth). Belief splits by percept after every move.
    Returns (worst-case plan length, belief states generated in the successful iteration)."""
    def split(bs):
        groups = {}
        for p in bs: groups.setdefault(percept(cells, p), set()).add(p)
        return [frozenset(g) for g in groups.values()]
    def solve(b, depth, memo):
        key = (b, depth)
        if key in memo: return memo[key]
        if b == {GOAL}: memo[key] = True; return True
        if depth == 0: memo[key] = False; return False
        memo[key] = False                                   # blocks cycles at same depth
        for a in ACT:
            nb = frozenset(map(move(cells, a), b))
            if nb == b: continue                            # no progress
            if all(solve(g, depth - 1, memo) for g in split(nb)):
                memo[key] = True; return True
        return False
    for D in range(0, max_depth + 1):
        memo = {}
        if all(solve(g, D, memo) for g in split(frozenset(start_set))):
            return D, len({k[0] for k in memo})
    return None, 0


def known_location(cells, start_set):
    """Original design (robot knows where it is): BFS from each start; plan length = worst start."""
    worst = 0
    for s in start_set:
        dist = {s: 0}; q = deque([s])
        while q:
            u = q.popleft()
            for a in ACT:
                v = move(cells, a)(u)
                if v not in dist: dist[v] = dist[u] + 1; q.append(v)
        worst = max(worst, dist[GOAL])
    return worst


if __name__ == "__main__":
    R = C = 6
    rows = []
    print(f"Floor {R}x{C} junctions with ~18% blocked; goal = (0,0). 5 floors (seeds 1-5) x 1 random start-set each.")
    print(f"{'k':>2} {'seed':>4} {'cells':>5} | {'known-loc plan':>14} | {'NO SENSOR beliefs':>17} {'plan':>5} |
{'SENSOR beliefs':>14} {'plan(worst)':>11} | {'2^N-1':>14}")
    for k in (2, 4, 8):
        for seed in range(1, 6):
            cells = make_floor(R, C, seed); rng = random.Random(100 * k + seed)
            starts = rng.sample(sorted(cells - {GOAL}), k)
            kl = known_location(cells, starts)
            pl, gen = sensorless(cells, starts)
            sp, sg = with_sensor(cells, starts)
            print(f"{k:>2} {seed:>4} {len(cells):>5} | {kl:>14} | {gen:>17,} {str(pl):>5} | {sg:>14,} {str(sp):>11} |
{2**len(cells)-1:>14,}")
            rows.append(dict(k=k, seed=seed, N=len(cells), known=kl, ns_gen=gen, ns_plan=pl, s_gen=sg, s_plan=sp))
    print("\nAverages per k:")
    print(f"{'k':>2} | {'known plan':>10} | {'no-sensor beliefs':>17} {'plan':>6} | {'sensor beliefs':>14}
{'plan':>6}")
    for k in (2, 4, 8):
        r = [x for x in rows if x["k"] == k]
        m = lambda key: statistics.mean([x[key] for x in r if x[key] is not None])
        print(f"{k:>2} | {m('known'):>10.1f} | {m('ns_gen'):>17,.0f} {m('ns_plan'):>6.1f} | {m('s_gen'):>14,.0f}
{m('s_plan'):>6.1f}")
    json.dump(rows, open("belief_results.json", "w"))
```

Console output of belief_search.py

```
Floor 6x6 junctions with ~18% blocked; goal = (0,0). 5 floors (seeds 1-5) x 1 random start-set each.
 k seed cells | known-loc plan | NO SENSOR beliefs  plan | SENSOR beliefs plan(worst) |          2^N-1
 2    1    29 |              9 |               328     9 |             18           9 |     536,870,911
 2    2    29 |              7 |               398     9 |             29           7 |     536,870,911
 2    3    31 |              3 |                51     3 |             14           3 |   2,147,483,647
 2    4    28 |              7 |               280     8 |             16           7 |     268,435,455
 2    5    28 |              5 |               136     6 |             13           5 |     268,435,455
 4    1    29 |              8 |             9,808    11 |             29           8 |     536,870,911
 4    2    29 |              9 |             5,283     9 |             28           9 |     536,870,911
```

```
 4    3    31 |              6 |             7,768     9 |             25           6 |  2,147,483,647
 4    4    28 |              7 |            13,338    12 |             26           7 |    268,435,455
 4    5    28 |              9 |             3,447    10 |             29           9 |    268,435,455
 8    1    29 |              9 |           101,873    12 |             31           9 |    536,870,911
 8    2    29 |             10 |            49,158    10 |             32          10 |    536,870,911
 8    3    31 |             10 |            45,840    10 |             28          10 |  2,147,483,647
 8    4    28 |              8 |            69,889    12 |             29           8 |    268,435,455
 8    5    28 |             13 |            26,175    14 |             31          13 |    268,435,455

Averages per k:
 k | known plan | no-sensor beliefs   plan | sensor beliefs   plan
 2 |        6.2 |               239    7.0 |             18    6.2
 4 |        7.8 |             7,929   10.2 |             27    7.8
 8 |       10.0 |            58,587   11.6 |             30   10.0
```

Summary of Task 12

Decision: MODIFY. The robot's knowledge becomes the state (belief state); the naive belief space is exponential ($2^N - 1$), but the reachable part is $O(N^k)$; search over belief states with a local sensor and a conditional plan generated 18–30 belief states for $k \leq 8$ against 239–58,587 without it, with plans as short as in the known-location case, and the original UCS is reused once the robot is localised.

Conclusion

The hospital robot was modelled as a rational, goal-based problem-solving agent and three different uninformed strategies were designed, analysed mathematically and tested on generated hospitals of 1,000 to 1,000,000 states. Theory and measurement agree: BFS and UCS grow linearly (times a log factor for UCS) and terminate after V expansions even for unreachable wards; the recurrence $T(L) = b \cdot T(L - 1) + 1$ was confirmed exactly on perfect trees; and IDDFS, although memory-light in its stack, repeats work and needs a duplicate table. Based on the evidence the recommended design is UCS with duplicate detection and a cached shortest-path tree from the pharmacy. When the localisation sensor fails the design is modified to search over belief states with a local sensor, and the original UCS is reused after the robot has localised itself.

References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020 – Chapters 1–4 (agents, problem solving, uninformed search, search with partial observations).

[2] R. E. Korf, "Depth-first iterative-deepening: an optimal admissible tree search," *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109, 1985.

[3] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022 (recurrences, graph algorithms).

Appendix A – experiments.py (full experiment driver)

```
"""Experiments for CSA17 Assignment 1  (Saai Vardhan D, 192525205)"""
import json, statistics, sys, time, random
from hospital_search import *


GRAPH_SEED = 42                 # graph generator seed
PAIR_SEEDS = [1, 2, 3]          # start/goal selection seeds
SIZES = [1000, 10000, 100000, 1000000]
RUNS = 5                        # timing repeats
out = {}


def pick(n, seed):
    r = random.Random(seed); s = r.randrange(n)
    g = r.randrange(n)
    while g == s: g = r.randrange(n)
    return s, g


def timed(f, adj, s, g, runs):
    ts = []
    for _ in range(runs):
        t = time.perf_counter(); res = f(adj, s, g); ts.append(time.perf_counter() - t)
    return res, statistics.mean(ts), (statistics.stdev(ts) if runs > 1 else 0.0)


# ------------------------------------------------------------------ TASK 8: functional tests
print("="*78); print("TEST SUITE  (graph seed 42, n = 1000 + sealed ward of 4 locations)"); print("="*78)
adj, e = build_graph(1000, GRAPH_SEED, sealed=4)
tests = [("reachable (pharmacy 17 -> ward 640)", 17, 640),
         ("unreachable (pharmacy 17 -> sealed ward 1003)", 17, 1003),
         ("start == goal (pharmacy 17 -> 17)", 17, 17)]
tr = []
for name, s, g in tests:
    print(f"\nTest: {name}")
    for an, f in ALGOS.items():
        p, c, st = f(adj, s, g)
        ok = (p is None) if g == 1003 else valid(adj, p, s, g)
        print(f"  {an:5s} path_len={'None' if p is None else len(p)-1:>4} cost={c!s:>4} gen={st.gen:>5} exp={st.exp:>5}
frontier={st.frontier:>4} valid/expected={ok}")
        tr.append(dict(test=name, algo=an, hops=None if p is None else len(p)-1, cost=c, gen=st.gen, exp=st.exp,
ok=ok))
out["tests"] = tr


# ------------------------------------------------------------------ TASK 8/9: scaling
print("\n" + "="*78); print("SCALING EXPERIMENT  (graph seed 42, pairs seeds 1,2,3, timing = mean of 5 runs on pair
1)"); print("="*78)
scal = []
for n in SIZES:
    t0 = time.perf_counter(); adj, e = build_graph(n, GRAPH_SEED); prep = time.perf_counter() - t0
    deg = sum(len(a) for a in adj) / n
    print(f"\nn={n:>8,}  edges={e:>8,}  avg degree={deg:.2f}  preprocessing(build adjacency)={prep:.3f}s")
    runs = RUNS
    rec = {a: dict(gen=[], exp=[], fr=[], hops=[], cost=[]) for a in ALGOS}
    times = {}
    for k, ps in enumerate(PAIR_SEEDS):
        s, g = pick(n, ps)
        for an, f in ALGOS.items():
            if k == 0:
                (p, c, st), tm, sd = timed(f, adj, s, g, runs); times[an] = (tm, sd)
            else:
                p, c, st = f(adj, s, g)
            assert valid(adj, p, s, g), (an, n)
            r = rec[an]; r["gen"].append(st.gen); r["exp"].append(st.exp); r["fr"].append(st.frontier)
            r["hops"].append(len(p)-1); r["cost"].append(c)
    ucs_cost = rec["UCS"]["cost"]; bfs_hops = rec["BFS"]["hops"]
    for an in ALGOS:
        r = rec[an]
        opt_cost = all(a == b for a, b in zip(r["cost"], ucs_cost))
        opt_hops = all(a == b for a, b in zip(r["hops"], bfs_hops))
        row = dict(n=n, algo=an, edges=e, deg=deg, prep=prep,
                   gen=statistics.mean(r["gen"]), exp=statistics.mean(r["exp"]), fr=statistics.mean(r["fr"]),
                   hops=r["hops"], cost=r["cost"], opt_cost=opt_cost, opt_hops=opt_hops,
                   t=times[an][0], tsd=times[an][1], gen_list=r["gen"], exp_list=r["exp"], fr_list=r["fr"])
        scal.append(row)
        print(f"  {an:5s} gen={row['gen']:>11,.0f} exp={row['exp']:>11,.0f} frontier={row['fr']:>9,.0f} "
              f"hops={r['hops']} cost={r['cost']} cost-opt={opt_cost} hop-opt={opt_hops} time={times[an][0]:.4f}s +-
{times[an][1]:.4f}")
    sys.stdout.flush()
    if n == 100000: adj100k = adj
    del adj
out["scaling"] = scal


# ------------------------------------------------------------------ TASK 4: best / average / worst (n = 100,000 + sealed
ward)
print("\n" + "="*78); print("BEST / AVERAGE / WORST CASE  (n = 100,000, seed 42, sealed ward added)"); print("="*78)
n = 100000
adj, e = build_graph(n, GRAPH_SEED, sealed=4)
s = 5
# BFS levels from s to find a farthest node
```

```
dist = {s: 0}; q = deque([s])
while q:
    u = q.popleft()
    for v, w in adj[u]:
        if v not in dist: dist[v] = dist[u] + 1; q.append(v)
far = max(dist, key=dist.get)
nb = min(adj[s], key=lambda x: x[1])[0]        # cheapest neighbour -> best case
cases = [("best: goal is neighbour of start", s, nb),
         ("average: random reachable goal (seed 1)", *pick(n, 1)),
         (f"worst reachable: farthest node (depth {dist[far]})", s, far),
         ("worst: goal unreachable (sealed ward)", s, n + 3)]
bw = []
for name, s_, g_ in cases:
    print(f"\n{name}")
    for an, f in ALGOS.items():
        try:
            p, c, st = (f(adj, s_, g_, cap=5_000_000) if an == "IDDFS" else f(adj, s_, g_))
        except Budget:
            print(f"  {an:5s} ABORTED - exceeded budget of 5,000,000 expansions without finishing")
            bw.append(dict(case=name, algo=an, hops=None, cost=None, gen=None, exp=">5,000,000 (aborted)", fr=None));
continue
        print(f"  {an:5s} hops={'None' if p is None else len(p)-1!s:>4} cost={c!s:>4} gen={st.gen:>8,} exp={st.exp:>8,}
frontier={st.frontier:>7,}")
        bw.append(dict(case=name, algo=an, hops=None if p is None else len(p)-1, cost=c, gen=st.gen, exp=st.exp,
fr=st.frontier))
out["cases"] = bw
del adj


# ------------------------------------------------------------------ unreachable-goal growth study
print("\n" + "="*78); print("UNREACHABLE GOAL: expansions vs graph size (seed 42, sealed ward)"); print("="*78)
gr = []
for n in (100, 200, 300, 400, 500, 600, 800):
    adj, e_ = build_graph(n, GRAPH_SEED, sealed=4)
    _, _, sb = bfs(adj, 0, n + 3); _, _, su = ucs(adj, 0, n + 3)
    try:
        _, _, si = iddfs(adj, 0, n + 3, cap=60_000_000); ie = si.exp
    except Budget: ie = None
    print(f"  n={n:>5}  BFS exp={sb.exp:>6}  UCS exp={su.exp:>6}  IDDFS exp={ie if ie else '>60,000,000':>12}")
    gr.append(dict(n=n, bfs=sb.exp, ucs=su.exp, iddfs=ie))
out["unreach"] = gr


# ------------------------------------------------------------------ TASK 6/7 validation: perfect b-ary tree, NO
duplicate detection
print("\n" + "="*78); print("RECURRENCE VALIDATION  (perfect b-ary tree, plain recursive DLS, no explored set)");
print("="*78)
def tree(b, depth):
    adj = [[]]; level = [0]
    for _ in range(depth):
        nl = []
        for u in level:
            for _ in range(b):
                adj.append([]); v = len(adj) - 1; adj[u].append((v, 1)); nl.append(v)
        level = nl
    return adj, level[-1]
def dls_tree(adj, u, g, L, cnt):
    cnt[0] += 1
    if u == g: return True
    if L == 0: return False
    for v, _ in adj[u]:
        if dls_tree(adj, v, g, L - 1, cnt): return True
    return False
val = []
for b in (2, 3):
    D = 10 if b == 2 else 7
    adj, g = tree(b, D)
    print(f"\nb={b}, tree depth {D}, goal = last leaf (worst case)")
    print(f"  {'L':>2} {'T(L) measured':>14} {'(b^(L+1)-1)/(b-1)':>20}")
    total = 0
    for L in range(0, D + 1):
        cnt = [0]; dls_tree(adj, 0, g, L, cnt); pred = (b ** (L + 1) - 1) // (b - 1)
        total += cnt[0]
        print(f"  {L:>2} {cnt[0]:>14} {pred:>20}")
        val.append(dict(b=b, L=L, measured=cnt[0], predicted=pred))
    pred_total = sum((b ** (L + 1) - 1) // (b - 1) for L in range(D + 1))
    print(f"  IDDFS total over limits 0..{D}: measured={total}  predicted={pred_total}  single DLS(d)={(b**(D+1)-
1)//(b-1)}  ratio={total/((b**(D+1)-1)//(b-1)):.3f}  (theory (b/(b-1))={b/(b-1):.3f})")
out["tree"] = val
json.dump(out, open("results.json", "w"), indent=1)
print("\nresults saved.")
```

Appendix B – Complete console output of experiments.py

```
================================================================================
TEST SUITE  (graph seed 42, n = 1000 + sealed ward of 4 locations)
================================================================================


Test: reachable (pharmacy 17 -> ward 640)
  BFS   path_len=   6 cost=  35 gen=  847 exp=  566 frontier= 344 valid/expected=True
  UCS   path_len=   8 cost=  18 gen=  687 exp=  329 frontier= 347 valid/expected=True
  IDDFS path_len=   6 cost=  35 gen= 2326 exp= 2332 frontier=  46 valid/expected=True


Test: unreachable (pharmacy 17 -> sealed ward 1003)
  BFS   path_len=None cost=None gen= 1000 exp= 1000 frontier= 344 valid/expected=True
  UCS   path_len=None cost=None gen= 1114 exp= 1000 frontier= 373 valid/expected=True
  IDDFS path_len=None cost=None gen=38859200 exp=38859637 frontier=1792 valid/expected=True


Test: start == goal (pharmacy 17 -> 17)
  BFS   path_len=   0 cost=   0 gen=    1 exp=    0 frontier=   0 valid/expected=True
  UCS   path_len=   0 cost=   0 gen=    1 exp=    1 frontier=   1 valid/expected=True
  IDDFS path_len=   0 cost=   0 gen=    1 exp=    1 frontier=   0 valid/expected=True


================================================================================
SCALING EXPERIMENT  (graph seed 42, pairs seeds 1,2,3, timing = mean of 5 runs on pair 1)
================================================================================


n=   1,000  edges=   1,499  avg degree=3.00  preprocessing(build adjacency)=0.001s
  BFS   gen=        709 exp=        479 frontier=      285 hops=[8, 7, 8] cost=[37, 33, 43] cost-opt=False hop-opt=True
time=0.0004s +-0.0001
  UCS   gen=      1,069 exp=        806 frontier=      358 hops=[11, 7, 12] cost=[34, 33, 35] cost-opt=True hop-
opt=False time=0.0009s +-0.0001
  IDDFS gen=      2,250 exp=      2,258 frontier=       53 hops=[8, 7, 8] cost=[37, 33, 43] cost-opt=False hop-opt=True
time=0.0018s +-0.0000


n=  10,000  edges=  14,998  avg degree=3.00  preprocessing(build adjacency)=0.016s
  BFS   gen=      4,096 exp=      2,790 frontier=    1,649 hops=[7, 7, 11] cost=[36, 32, 60] cost-opt=False hop-
opt=True time=0.0005s +-0.0002
  UCS   gen=      6,302 exp=      4,160 frontier=    2,801 hops=[8, 10, 12] cost=[28, 29, 46] cost-opt=True hop-
opt=False time=0.0030s +-0.0003
  IDDFS gen=     13,899 exp=     13,907 frontier=       78 hops=[7, 7, 11] cost=[36, 32, 60] cost-opt=False hop-
opt=True time=0.0010s +-0.0001


n= 100,000  edges= 149,998  avg degree=3.00  preprocessing(build adjacency)=0.189s
  BFS   gen=     55,949 exp=     33,602 frontier=   23,902 hops=[12, 8, 11] cost=[73, 46, 62] cost-opt=False hop-
opt=True time=0.1471s +-0.0127
  UCS   gen=    109,606 exp=     90,245 frontier=   36,974 hops=[14, 8, 15] cost=[60, 46, 53] cost-opt=True hop-
opt=False time=0.5116s +-0.0192
  IDDFS gen=    155,802 exp=    155,813 frontier=      103 hops=[12, 8, 11] cost=[73, 46, 62] cost-opt=False hop-
opt=True time=0.4521s +-0.0056


n=1,000,000  edges=1,499,995  avg degree=3.00  preprocessing(build adjacency)=2.840s
  BFS   gen=    732,720 exp=    504,611 frontier=  283,320 hops=[13, 15, 12] cost=[78, 76, 66] cost-opt=False hop-
opt=True time=2.2091s +-0.1093
  UCS   gen=  1,038,308 exp=    768,379 frontier=  368,627 hops=[16, 20, 14] cost=[59, 58, 60] cost-opt=True hop-
opt=False time=9.0114s +-0.2147
  IDDFS gen=  2,316,984 exp=  2,316,997 frontier=      154 hops=[13, 15, 12] cost=[78, 76, 66] cost-opt=False hop-
opt=True time=6.5635s +-0.5001


================================================================================
BEST / AVERAGE / WORST CASE  (n = 100,000, seed 42, sealed ward added)
================================================================================


best: goal is neighbour of start
  BFS   hops=   1 cost=   1 gen=       3 exp=       1 frontier=      1
  UCS   hops=   1 cost=   1 gen=       5 exp=       2 frontier=      4
  IDDFS hops=   1 cost=   1 gen=       3 exp=       4 frontier=      4


average: random reachable goal (seed 1)
  BFS   hops=  12 cost=  73 gen=  87,096 exp=  60,463 frontier= 31,298
  UCS   hops=  14 cost=  60 gen= 111,965 exp=  99,270 frontier= 36,981
  IDDFS hops=  12 cost=  73 gen= 290,428 exp= 290,440 frontier=    113


worst reachable: farthest node (depth 13)
  BFS   hops=  13 cost=  75 gen=  99,956 exp=  99,434 frontier= 31,467
  UCS   hops=  13 cost=  57 gen= 111,961 exp=  99,850 frontier= 36,876
  IDDFS hops=  13 cost=  75 gen= 923,302 exp= 923,315 frontier=    139


worst: goal unreachable (sealed ward)
  BFS   hops=None cost=None gen= 100,000 exp= 100,000 frontier= 31,467
  UCS   hops=None cost=None gen= 111,969 exp= 100,000 frontier= 36,876
  IDDFS ABORTED - exceeded budget of 5,000,000 expansions without finishing


================================================================================
```

```
UNREACHABLE GOAL: expansions vs graph size (seed 42, sealed ward)
================================================================================
  n=  100  BFS exp=   100  UCS exp=   100  IDDFS exp=       32219
  n=  200  BFS exp=   200  UCS exp=   200  IDDFS exp=      251757
  n=  300  BFS exp=   300  UCS exp=   300  IDDFS exp=     1006899
  n=  400  BFS exp=   400  UCS exp=   400  IDDFS exp=     2592609
  n=  500  BFS exp=   500  UCS exp=   500  IDDFS exp=     4596207
  n=  600  BFS exp=   600  UCS exp=   600  IDDFS exp=     9103155
  n=  800  BFS exp=   800  UCS exp=   800  IDDFS exp=    19680215


================================================================================
RECURRENCE VALIDATION  (perfect b-ary tree, plain recursive DLS, no explored set)
================================================================================

b=2, tree depth 10, goal = last leaf (worst case)
   L  T(L) measured    (b^(L+1)-1)/(b-1)
   0              1                    1
   1              3                    3
   2              7                    7
   3             15                   15
   4             31                   31
   5             63                   63
   6            127                  127
   7            255                  255
   8            511                  511
   9           1023                 1023
  10           2047                 2047
  IDDFS total over limits 0..10: measured=4083  predicted=4083  single DLS(d)=2047  ratio=1.995  (theory (b/(b-
1))=2.000)

b=3, tree depth 7, goal = last leaf (worst case)
   L  T(L) measured    (b^(L+1)-1)/(b-1)
   0              1                    1
   1              4                    4
   2             13                   13
   3             40                   40
   4            121                  121
   5            364                  364
   6           1093                 1093
   7           3280                 3280
  IDDFS total over limits 0..7: measured=4916  predicted=4916  single DLS(d)=3280  ratio=1.499  (theory (b/(b-
1))=1.500)

results saved.
```

Appendix C – Measurement of the final depth limit for unreachable goals (lstar.py output)

```
     n BFS ecc of start   final limit L* (IDDFS)   IDDFS exp   V*L*^2/2
  100                5                      39      32,219     76,050
  200                5                      80     251,757    640,000
  400                7                     181   2,592,609  6,552,200
  800                9                     348  19,680,215 48,441,600
```

Appendix D – Additional computations (extra.py output)

```
== IDDFS duplicate-table size (final iteration, pair seed 1) vs BFS parent-table size ==
n=   1,000 limit d= 8 IDDFS seen-table entries=      946 BFS parent entries(gen)=      965 IDDFS stack frontier=56
n=  10,000 limit d= 7 IDDFS seen-table entries=      751 BFS parent entries(gen)=    1,607 IDDFS stack frontier=69
n= 100,000 limit d=12 IDDFS seen-table entries=   71,351 BFS parent entries(gen)=   87,096 IDDFS stack frontier=113
n=1,000,000 limit d=13 IDDFS seen-table entries=  834,438 BFS parent entries(gen)=  884,195 IDDFS stack frontier=183


== log-log growth exponent between n=1e4 and 1e6 (slope = log10(ratio)/2) ==
nodes generated  BFS    ratio=     178.9 slope= 1.13
nodes generated  UCS    ratio=     164.7 slope= 1.11
nodes generated  IDDFS  ratio=     166.7 slope= 1.11
nodes expanded   BFS    ratio=     180.9 slope= 1.13
nodes expanded   UCS    ratio=     184.7 slope= 1.13
nodes expanded   IDDFS  ratio=     166.6 slope= 1.11
max frontier     BFS    ratio=     171.8 slope= 1.12
max frontier     UCS    ratio=     131.6 slope= 1.06
max frontier     IDDFS  ratio=       2.0 slope= 0.15
time (1e5->1e6)  BFS    ratio=      15.0 slope= 1.18
time (1e5->1e6)  UCS    ratio=      17.6 slope= 1.25
time (1e5->1e6)  IDDFS  ratio=      14.5 slope= 1.16


== effective branching b* = gen^(1/d)  (d = mean BFS hops) ==
n=   1,000 d= 7.67 gen=       709 b*=2.35
n=  10,000 d= 8.33 gen=     4,096 b*=2.71
n= 100,000 d=10.33 gen=    55,949 b*=2.88
n=1,000,000 d=13.33 gen=   732,720 b*=2.75


== per-expansion time (microseconds) at n=1e6 and ratios vs BFS expansions ==
```

```
BFS    3.51 us per expansion (pair-1 expansions=629,360)
UCS    10.35 us per expansion (pair-1 expansions=870,564)
IDDFS  2.25 us per expansion (pair-1 expansions=2,920,777)
n=   1,000 IDDFS/BFS expansions = 4.72  UCS/BFS expansions=1.68  UCS/BFS gen=1.51  UCS/BFS time=2.2
n=  10,000 IDDFS/BFS expansions = 4.98  UCS/BFS expansions=1.49  UCS/BFS gen=1.54  UCS/BFS time=5.5
n= 100,000 IDDFS/BFS expansions = 4.64  UCS/BFS expansions=2.69  UCS/BFS gen=1.96  UCS/BFS time=3.5
n=1,000,000 IDDFS/BFS expansions = 4.59  UCS/BFS expansions=1.52  UCS/BFS gen=1.42  UCS/BFS time=4.1


== belief-space size bound: number of subsets of size <= k out of N=29 cells ==
2 435   2^29-1 = 536870911
4 27840   2^29-1 = 536870911
8 6474540   2^29-1 = 536870911
```